

Eco Forecast

Final Year Project

Session 2022-2026

A project submitted in partial fulfillment of the degree of

BS in Computer Science



Department of Computer Science

Faculty of Computer Science & Information Technology

The Superior University, Lahore

Fall 2025

Type (Nature of project)	[☒] Development [☐] Research [☐] R&D			
Area of specialization	Artificial Intelligence, Deep Learning and Explainable AI			
FYP ID	FYP-BSCS-F25-06			
Project Group Members				
Sr.#	Reg. #	Student Name	Email ID	*Signature
(i)	Su92-Bscsm-F22-133	M Saqib Masood (Group Leader)	Su92_bscsm_f22_133	
(ii)	Su92-Bscsm-F22-130	M Hamza	Su92_bscsm_f22_130	
(iii)	Su92-Bscsm-F22-137	Laiba Ali	Su92_bscsm_f22_137	

Plagiarism Free Certificate

This is to certify that I, **Saqib Masood**, group leader of FYP under registration no **Su92-Bscsm-F22-133** at Computer Science Department, The Superior University, Lahore. I declare that my FYP report is checked by my supervisor.

Date: _____ Name of Group Leader: Saqib Masood Signature: _____

Name of Supervisor: Sohaib Hafeez

Co-Supervisor: Syed Karar Haider Bukhari

Designation: Lecturer

Designation: Lecturer

Signature: _____

Signature: _____

HOD: Azam chaudhary

Signature: _____

Project Report

Smart Energy Consumption Forecasting with Explainable AI Integration

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature
M Saqib Masood, M Hamza, Laiba Ali	1.0	20 September 2025	Original Draft	
M Saqib Masood, M Hamza, Laiba Ali	1.1	11 Oct 2025	Changes Based on Feedback from Supervisor	
M Saqib Masood, M Hamza, Laiba Ali	1.2	25 November 2025	Changes Based on Feedback From Faculty	
M Saqib Masood, M Hamza, Laiba Ali	1.3	13 December 2025	Added Project Plan	

APPROVAL

PROJECT SUPERVISOR

Comments: _____

Name: _____

Date: _____ Signature: _____

PROJECT MANAGER

Comments: _____

Date: _____ Signature: _____

HEAD OF THE DEPARTMENT

Comments: _____

Date: _____ Signature: _____

Dedication

In the name of Allah, the Most Gracious, the Most Merciful.

This Final Year Project is wholeheartedly dedicated to our beloved parents, whose unconditional love, endless prayers, and extraordinary sacrifices have been our greatest source of strength throughout our academic journey. Without their unwavering belief in our abilities and their tireless encouragement through every challenge, this achievement would not have been possible. Every milestone we reach is a reflection of the values they have instilled in us.

We dedicate this work to our respected teachers and mentors at The Superior University, Lahore, particularly our supervisor Mr. Sohaib Hafeez and co-supervisor Mr. Syed Karar Haider Bukhari, whose expert guidance, intellectual insight, and patient mentorship shaped not only this project but also our understanding of research and engineering. Their dedication to our growth as students and professionals has left a lasting impression on us.

We also dedicate this effort to every member of our project team M Saqib Masood, M Hamza, and Laiba Ali whose collaboration, mutual support, and shared commitment transformed this project from an idea into a reality. The long hours of problem-solving, debugging, and discussion were made meaningful by the spirit of teamwork we shared.

Finally, we dedicate this work to the people of Pakistan particularly those who face the daily hardships of electricity shortages and load shedding. It is our sincere hope that the work presented in this report, Eco Forecast, contributes in some meaningful way toward a more stable, efficient, and data-driven energy future for our nation. May this project serve as a small step toward smarter, greener, and more sustainable energy management in Pakistan.

Acknowledgements:

All praise and gratitude belong to Almighty Allah, the Most Gracious and the Most Merciful, who bestowed upon us the knowledge, patience, strength, and perseverance required to complete this Final Year Project. Without His blessings and guidance at every step of this journey, none of this would have been achievable.

We express our deepest and most sincere gratitude to our supervisor, Mr. Sohaib Hafeez (Lecturer, Department of Computer Science, The Superior University, Lahore), whose consistent guidance, constructive feedback, and intellectual mentorship were invaluable throughout the entire development of Eco Forecast. His ability to challenge our thinking while supporting our progress made this project significantly stronger. We are truly grateful for the time and expertise he generously devoted to us.

We are equally thankful to our co-supervisor, Mr. Syed Karar Haider Bukhari (Lecturer, Department of Computer Science, The Superior University, Lahore), for his technical insights, encouragement, and support during the implementation and documentation phases of the project. His contributions helped us navigate complex challenges with greater clarity and confidence.

Our sincere appreciation also goes to the Head of Department, Mr. Azam Chaudhary, and the entire faculty of the Department of Computer Science at The Superior University, whose academic guidance and institutional support provided the foundation for our research. The department's commitment to quality education and research excellence has been a constant source of inspiration.

We would also like to acknowledge the open-source community behind the tools and frameworks used in this project including TensorFlow, Streamlit, SHAP, LIME, and LangChain whose contributions made advanced AI development accessible to students and researchers worldwide.

Last but not least, we owe a heartfelt thank you to our families, friends, and classmates who stood by us with encouragement, understanding, and moral support throughout the highs and lows of this project. Your belief in us kept us going when the journey felt difficult.

Executive Summary

Pakistan faces one of the most critical energy challenges in its history. Rapid urbanization, extreme seasonal weather variability, inconsistent household consumption patterns, and aging grid infrastructure have collectively produced a highly volatile electricity demand landscape. Traditional forecasting methods such as ARIMA and Exponential Smoothing are no longer adequate to capture the non-linear and dynamic nature of this demand, resulting in frequent power outages, inefficient grid operations, and significant economic losses for both utilities and consumers.

Eco Forecast is an AI-powered, short-term electricity demand forecasting platform developed as a Final Year Project by students of the Department of Computer Science at The Superior University, Lahore. The system directly addresses Pakistan's electricity forecasting challenges by combining advanced deep learning architectures with Explainable AI (XAI) and natural language interaction, all delivered through an accessible web interface.

The core forecasting engine integrates three complementary deep learning architectures: Convolutional Neural Networks (CNN) for extracting local temporal patterns, Long Short-Term Memory networks (LSTM) for capturing long-range sequential dependencies, and Gated Recurrent Units (GRU) for efficient sequential modeling. These models are trained on multi-source data including the REWDP household consumption dataset and real-time weather data from the Open-Meteo API. The ensemble of CNN, LSTM, and GRU achieved an R^2 score of 0.934 and RMSE of 0.328 kWh – a 19.1% improvement over the traditional ARIMA baseline.

A key differentiator of Eco Forecast is its Explainable AI integration. Using SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-Agnostic Explanations), the system provides both global feature importance visualizations – showing which variables like temperature, hour of day, and appliance load most influence forecast outcomes – and local instance-level explanations for individual predictions. This transparency builds trust among utility analysts, grid operators, and household users who need to act on the forecasts.

The system also incorporates a Retrieval-Augmented Generation (RAG) powered intelligent Q&A agent built with LangChain and FAISS. This agent allows users to query forecast insights in natural language – for example, asking why electricity demand is expected to peak at a

specific time and receive contextually accurate, human-readable responses without requiring any technical expertise.

The entire platform is deployed as a Streamlit web application, accessible on standard hardware with a minimum of 8 GB RAM. The application features an interactive Forecast Panel, an Explainability Panel with SHAP and LIME visualizations, a Smart Q&A chat interface, and an Export module for PDF and CSV report generation. Comprehensive testing including use case testing, equivalence partitioning, boundary value analysis, unit testing, integration testing, performance testing, and stress testing confirms that all functional and non-functional requirements are met. Eco Forecast is open-source, Pakistan-specific, and designed to empower both energy utilities and everyday consumers with accurate, interpretable, and actionable electricity demand insights.

Contents

Final Year Project	i
Session 2022-2026.....	i
Plagiarism Free Certificate	iii
Dedication	vi
Acknowledgements:.....	vii
Executive Summary	viii
List of Figures	xiv
List of Tables	xvi
Chapter 1	2
Introduction.....	2
Chapter 1: Introduction:.....	3
1.1 Background	3
1.2 Motivations and Challenges.....	3
1.3 Literature Review/Existing Solutions	4
1.4 Gap Analysis	4
1.5 Propsoed Solution	5
1.6 Project Plan	5
1.6.1 Work Breakdown Structure	6
1.6.2 Roles & Responsibility Matrix	7
1.6.3 Gantt Chart.....	8
1.7 Report Outline.....	8
Chapter 2.....	9
Software Requirement Specifications	9
Chapter 2: Software Requirement Specifications	10
2.1 Introduction.....	10
2.1.1 Purpose.....	10
2.1.2 Document Conventions.....	10
2.1.3 Intended Audience and Reading Suggestions.....	11
2.1.4 Product Scope	11
2.1.5 References.....	11
2.2 Overall Description.....	12
2.2.1 Product Perspective.....	12
2.2.2 Product Functions	13
2.2.3 User Classes and Characteristics	13
2.2.4 Operating Environment.....	13
2.2.5 Design and Implementation Constraints.....	13
2.2.6 User Documentation	14
2.2.7 Assumptions and Dependencies	14
2.3 External Interface Requirements.....	14

2.3.1 User Interfaces	14
2.3.2 Hardware Interfaces	15
2.3.3 Software Interfaces	15
2.3.4 Communications Interfaces	15
2.4 System Features	15
2.4.1 System Feature 1.....	15
2.4.2 System Feature 2.....	15
2.4.3 System Feature 3:.....	16
2.5 Other Nonfunctional Requirements	16
2.6 Other Requirements	17
Chapter 3.....	18
Use Case Analysis.....	18
Chapter 3: Use Case Analysis.....	19
3.1. Use Case Model	19
3.2 Use Case Descriptions	20
Chapter 4.....	30
System Design	30
Chapter 4: System Design.....	31
4.1 Architecture Diagram.....	32
4.2 Entity Relationship Diagram with data dictionary.....	33
4.3 Class Diagram	34
4.4 Sequence / Collaboration Diagram	35
4.5 Operation contracts	35
4.6 Activity Diagram	36
4.7 State Transition Diagram	36
4.8 Component Diagram	37
4.9Deployment Diagram.....	38
4.10 Data Flow diagram.....	39
Chapter 5.....	40
Implementation	40
Chapter 5: Implementation	41
5.1. Important Flow Control/Pseudo codes	41
5.2. Components, Libraries, Web Services and stubs	42
5.3. Deployment Environment	43
5.4. Tools and Techniques.....	43
5.5. Best Practices & Folder Structure	43
5.6. Version Control.....	44
Chapter 6.....	45
Testing and Evaluation	45
6.1 Use Case Testing.....	46
6.2 Equivalence Partitioning.....	47

6.3 Boundary Value Analysis	48
6.4 Data Flow Testing.....	50
6.4 Data Flow Paths	50
6.5 Unit Testing	52
6.5 Data Loader Module	52
6.6 Model Manager Module	53
6.7 XAI Engine Module.....	53
6.8 RAG Engine Module	54
6.9 Integration Testing.....	54
6.10 Performance Testing	55
6.11 Model Accuracy Metrics.....	56
6.12 Stress Testing.....	57
6.8.1 Summary of Test Results	58
Chapter 7	59
Summary, Conclusion and Future Enhancements	59
7.1 Project Summary.....	60
7.2 Achievements and Improvements.....	61
7.1 Technical Achievements	61
7.2.2 Improvements Over Existing Solutions	61
7.3 Critical Review	62
7.3.1 Strengths	62
7.3.2 Limitations	62
7.4 Lessons Learnt	63
7.4.1 Technical Lessons.....	63
7.4.2 Project Management Lessons	64
7.5 Future Enhancements / Recommendations.....	64
7.2 High Priority Enhancements	64
7.3 Medium Priority Enhancements	65
7.5.3 Low Priority / Research Extensions.....	66
Appendix.....	68
Appendix: User Manual.....	69
A.1 System Requirements.....	69
A.2 Installation and Setup.....	69
A.2.1 Clone the Repository.....	69
A.2.2 Install Dependencies	70
A.2.3 Configure Environment Variables	70
A.2.4 Launch the Application.....	70
A.3 Navigating the Dashboard.....	70
A.3.1 Login Screen	70
A.3.2 Main Dashboard Layout.....	71
A.4 Generating a Forecast.....	71
A.5 Viewing Explainability Reports.....	71
A.6 Using the Smart Q&A Agent	72
A.7 Troubleshooting Common Issues.....	72

Appendix B: Administrator Manual	73
B.1. Administrator Responsibilities	73
B.2. Accessing the Admin Panel	73
B.3. User Management	73
B.3.1. Adding a New User	73
B.3.2. Deactivating a User	74
B.4. Dataset Management	74
B.4.1. Uploading a New Dataset	74
B.4.2. Archiving Old Datasets	74
B.5. Model Retraining	74
B.6. Backup and Recovery	75
B.6.1. Manual Backup	75
B.6.2. Restoring from Backup	75
B.7. Monitoring and Logs	75
Appendix C: Information / Promotional Material	77
C.1. Project Poster	77
C.1.1. Poster Title and Tagline	77
C.1.2. Poster Content Summary	77
C.2. Project Brochure	78
C.2.1. Front Panel	78
C.2.2. Inner Panel Key Features	78
C.2.3. Back Panel Team and Contact	78
C.3. Project Abstract Card	79
Appendix D: Glossary of Terms	80
Bibliography & References	82
Bibliography & References	83

List of Figures

1.1 Gantt Chart	8
2.1 Eco Forecast System	12
3.1 Use Case Diagram	21
4.1 Architecture Diagram	32
4.2 Entity Relationship Diagram with data dictionary	33
4.3 Class Diagram	34
4.4 Sequence / Collaboration Diagram	35
4.5 Operation contracts	35
4.6 Activity Diagram	36
4.7 State Transition Diagram	36
4.8 Component Diagram	37
4.9 Deployment Diagram	38
4.10 Data Flow Diagram	40

List of Tables

1.1 Work Breakdown Structure	6
1.2 Roles & Responsibility Matrix	7
2.1 User Classes and Characteristics	13
3.1 Fully Dressed Use Case 1	22
3.2 Fully Dressed Use Case 2	23
3.3 Fully Dressed Use Case 3	24
3.4 Fully Dressed Use Case 4	25
3.5 Fully Dressed Use Case 5	26
3.6 Fully Dressed Use Case 6	27
3.7 Fully Dressed Use Case 7	28
3.8 Fully Dressed Use Case 8	29
5.3 Deployment Environment	43
6.1 Use Case Testing	47
6.2 Equivalence Partitioning	48
6.3 Boundary Value Analysis	49
6.4 Data Flow Paths	50
6.5 Data Loader Module	52
6.6 Model Manager Module	53
6.7 XAI Engine Module	53
6.8 RAG Engine Module	54
6.9 Integration Testing	54
6.10 Performance Testing	55
6.11 Model Accuracy Metrics	56
6.12 Stress Testing	57
7.1 Technical Achievements	61
7.2 High Priority Enhancements	64
7.3 Medium Priority Enhancements	65

Chapter 1

Introduction

Chapter 1: Introduction:

The chapter gives a summary of the project Eco Forecast: Smart Energy Consumption Forecasting with Explainable AI Integration. It provides the background and context of electricity demand forecasting, outlines the issues the Pakistani energy sector is encountering, and justifies the reasons why it is necessary to adopt advanced Artificial Intelligence methods. The chapter is also used to list the goals, objectives, and scope of the study and briefly discusses the existing solutions and limitations. Lastly, it outlines the proposed solution, project plan and structure of the report, which forms the basis of the other chapters.

1.1 Background

Proper operation of power systems, planning of tariffs, and management of demand side, depend on the energy demand forecasting. In Pakistan, an extreme weather variability, high urbanization rate, and intermittent household behavior contribute to the dramatic changes in demand, posing challenges to the conventional approaches (e.g., ARIMA, Holt Winters). The new deep learning models CNN to extract local patterns, LSTM/GRU to extract long range temporal dependencies- better perform on complex nonlinear time series. However, an interpretability is a major roadblock to adoption. Explainable AI (XAI) is a solution to this by labeling the predictions based on features (e.g., temperature, hour of the day) as well as revealing model behavior to the stakeholders.

1.2 Motivations and Challenges

Motivations:

Decimate blackouts through better short term predictions.

Facilitate consumer savings by interpretable directive;

Help utilities with planning and scheduling;

Have an academic standard that incorporates accuracy and explain ability.

Challenges:

Multi source data integration (weather, house, appliance signals);

CNN/LSTM/GRU Hyper parameter optimization;
 Generalization in extreme weather conditions;
 Tradeoff between accuracy and interpretability;
 Responsive, efficient web deployment Goals and Objectives.

1.3 Literature Review/Existing Solutions

The investigation of energy demand has long been a topic of research, and early models have been based on classical statistical models, like ARIMA and Exponential Smoothing, which give reasonable results in the short term, but fail with nonlinear and highly dynamic consumption trends [1]. With the rise in demand variation through urbanization and dynamic weather conditions, modelers started implementing machine learning tools, including Support Vector Regression (SVR) and random forests, capable of performing greater generalization, but not the ability to respond to long-term dependence on the time series [2].

The introduction of deep learning gave birth to time-series forecasting architectures, including Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), which achieved better results because they were able to capture sequential dependencies. Some of the studies used LSTM-based models to data on electricity demands and noted that they made significant gains compared with the conventional ones [3]. More recently, hybrid models with Convolutional Neural Networks (CNN) and recurrent layers to extract both spatial and temporal features to enhance forecasting accuracy have also been presented [4].

In spite of these achievements, the majority of the current literature considers deep learning models as black boxes, which do not provide end-users or utility providers with much interpretability. This is not transparent enough, which minimizes trust and restricts the real-life application. To solve this, new studies on Explainable AI (XAI) through approaches such as SHAP and LIME-have been incorporated into the forecasting models to offer feature-level information and enhance the trust of stakeholders [5].

There is a lack of similar studies that integrate multi-model deep learning forecasting with XAI integration and web deployment in real-time, though, which is what our project aims to provide.

1.4 Gap Analysis

Precision vs. disclosure: Most successful model types are hardly interpretable.

End to end integration: There is not a lot of systems where multi model DL, weather fusion, XAI, and web deployment are brought together in a single pipeline.

Action ability: Current dashboards usually end at plots; actionable advice through RAG/Q&A is seldom found.

Local context: Few studies focus on the demand pattern and data idiosyncrasy of Pakistan.

1.5 Propsoed Solution

Eco Forecast incorporates multi source data (REWDP, weather APIs) in combination with a hybrid DL stack (CNN + LSTM + GRU). It gives short term demand forecasts, with confidence and visualizations (XAI) of feature importance globally (global feature importance) and locally (local explanations of specific forecasts) features. An application based on Streamlit delivers insights via a smart RAG agent to query natural language (e.g. why is the demand higher tomorrow at 7-9 pm). The architecture puts modularity first, making it easy to study ablation and extend it in the future (e.g. attention mechanisms, transformer baselines).

1.6 Project Plan

The proposed project is to be done in several stages, approximately over two regular semesters. The phases will concentrate on the essential area of system development, which will start with data collection and finish with deployment and documentation. The plan facilitates systematic advancement by setting up, execution, testing and eventually reporting.

The stages can be described as the following:

Week 1-2: Gathering of requirements, definition of problem, and selection of dataset.

Weeks 3-4: Data collection, cleaning, feature engineering (weather, household, appliance data).

Weeks 5-8: Models design and development (CNN, LSTM, GRU); comparison with baseline models such as statistical models.

Weeks 9-10: Explainable AI (SHAP, LIME, attention visualizations).

Weeks 11-14: Building Streamlit-based web app and embedding of Q&A agent (RAG).

Weeks 15-16: Testing, evaluation, performance validation.

Weeks 17-18: Final documentation, report writing and submission.

This plan will make sure that the project can be accomplished within the period of time given and balance both technical implementation and academic reporting.

1.6.1 Work Breakdown Structure

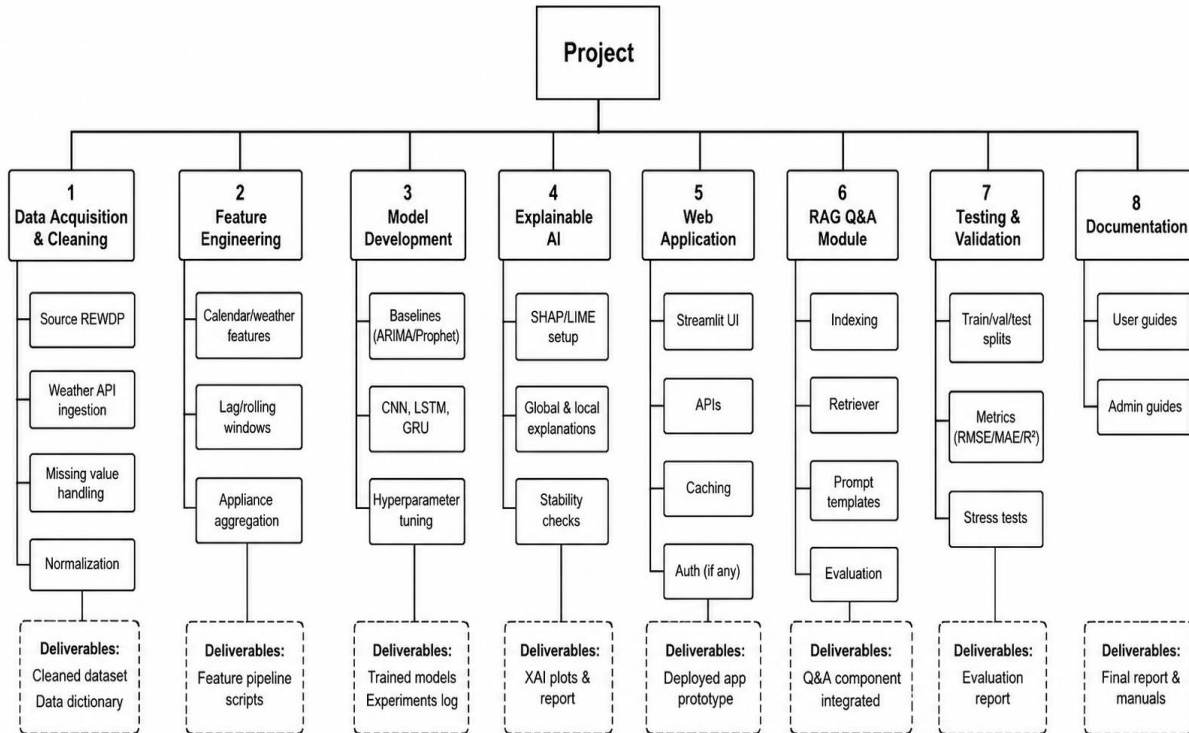


Table 1.1: Work Breakdown Structure

1.6.2 Roles & Responsibility Matrix

Task / Phase	Saqib Masood (Lead)	M Hamza	Laiba Ali
	Lead Developer	Developer	Analyst
Data Acquisition & Cleaning	A/R	C	C
Feature Engineering	A/R	R	C
Model Development	A/R	R	C
Explainable AI	A/R	C	I
Web Application	A/R	A/R	A/R
RAG Q&A Module	A/R	R	C
Testing & Validation	A/R	R	C
Documentation	A/R	C	I

Table No. 1.2

R = Responsible - The actual worker.

A = Accountable - The same person who owns the task and makes sure that it is done properly. (A is normally one person per task).

C = Consulted - The individual who provides input, advice or expertise. They consulted in the task and do not do that directly.

I = Informed - This person is informed of progress and not directly engaged.

1.6.3 Gantt Chart

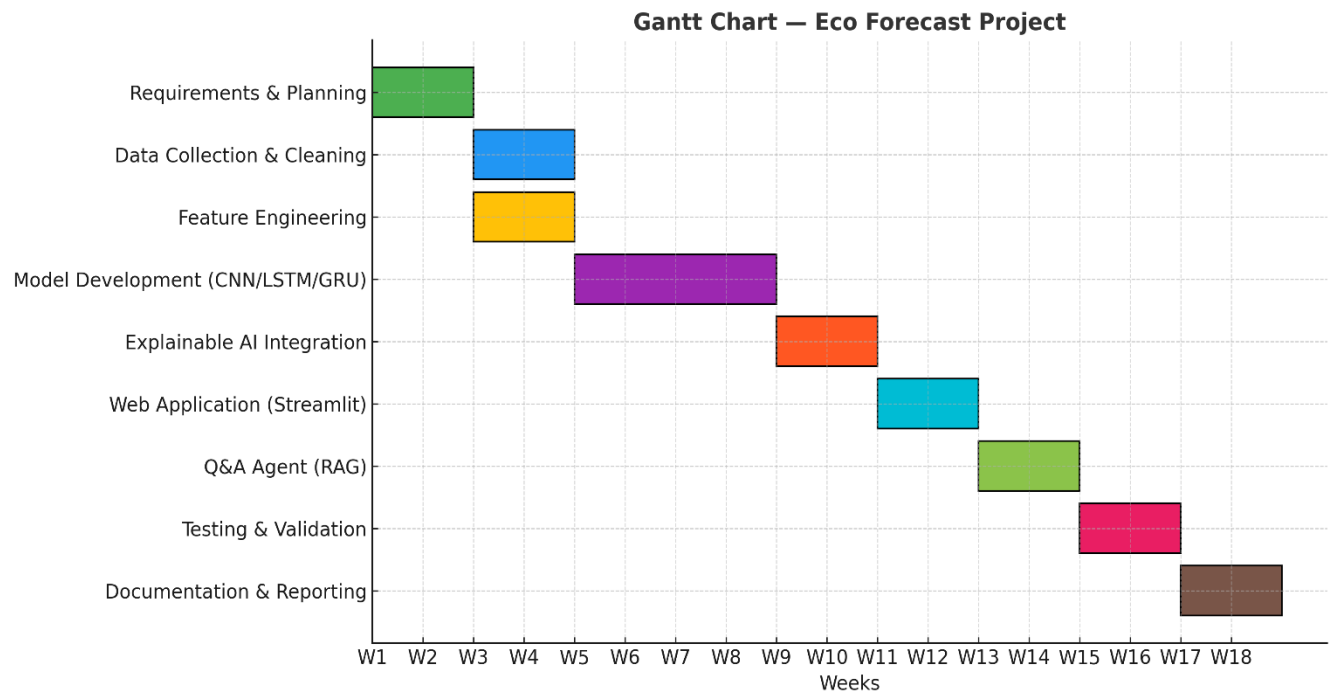


Figure No Gantt Chart. 1.1

1.7 Report Outline

Chapter 1: Introduction - background, problem statement, objectives and literature review, gap analysis, proposed solution and project plan.

Chapter 2: Software Requirements Specification- system scope, requirements, and constraints.

Chapter 3: Use Case Analysis - use case model and descriptions.

Chapter 4: System Design - architecture, data models and UML diagrams.

Chapter 5: Implementation - software, technologies, and code practices.

Chapter 6: Testing and Evaluation - testing techniques, validation and performance analysis.

Chapter 7: Conclusion and Future Work - conclusion, findings, and future improvements.

Chapter 2

Software Requirement Specifications

Chapter 2: Software Requirement Specifications

2.1 Introduction

This chapter presents the Software Requirements Specification (SRS) for Eco Forecast, a Streamlit-based web application for short-term electricity demand forecasting in Pakistan. It defines the system's functional and non-functional requirements, interfaces, constraints, and assumptions in accordance with the IEEE 830-1998 standard. The chapter provides a clear understanding of system scope, user interactions, and operational requirements to guide development and evaluation.

2.1.1 Purpose

This SRS document defines the software requirements for Eco Forecast, version 1.0 a Streamlit-based web application designed for short-term electricity demand forecasting in Pakistan using hybrid deep learning models (CNN, LSTM, GRU) and Explainable AI (XAI) methods such as SHAP and LIME.

It specifies the functional and non-functional requirements, design constraints, interfaces, and dependencies necessary for development, deployment, and maintenance. This document covers the entire system scope, including the forecasting engine, XAI visualization module, and the RAG-based intelligent Q&A agent.

2.1.2 Document Conventions

This document follows IEEE 830-1998 SRS guidelines.

Font Style: Headings – Bold Calibri 14; Body – Calibri 11.

Requirement Identification: Each functional requirement is labeled (e.g., REQ-SF1-1).

Priorities:

High – Core system functionality.

Medium – Optional but beneficial features.

Low – Future extensions or research add-ons.

Keywords such as *must*, *should*, *may* denote requirement strength (mandatory, recommended, optional).

2.1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

Project Supervisors / Evaluators – To understand system objectives, scope, and compliance with academic and IEEE SRS standards.

Developers and Data Scientists – To use the requirements as a reference for system design, implementation, and model development.

Utility Analysts and Energy Experts – To understand how forecasting and explainability features support grid planning.

Future Researchers – To extend the system for further experimentation and enhancements.

Reading Suggestions:

Readers seeking a **high-level overview** should read Sections **2.1 and 2.2**.

Developers should focus on **Sections 2.2, 2.3, and Chapter 3 (Functional Requirements)**.

Evaluators may review the entire document sequentially for full system understanding.

2.1.4 Product Scope

Eco Forecast aims to deliver accurate short-term demand predictions and interpretable insights to enhance grid stability and assist consumers in energy management.

The system integrates multi-source data (REWDP + weather API + household usage), hybrid DL models, and XAI visualization into one unified web platform.

Objectives:

Provide transparent and interpret-able energy forecasting.

Enable grid operators to plan load balancing efficiently.

Help households optimize consumption and cost.

Promote data-driven decision-making in Pakistan’s energy sector.

2.1.5 References

[1] Chollet, F. Deep Learning with Python. Manning, 2021.
https://scholar.google.com.pk/scholar?q=%5D+Chollet,+F.+Deep+Learning+with+Python.+Manning,+2021.&hl=en&as_sdt=0&as_vis=1&oi=scholart

[2] Lundberg & Lee (2017). “A Unified Approach to Interpreting Model Predictions,” NeurIPS.

- [3] Streamlit Documentation (2024). <https://docs.streamlit.io/>
- [4] SHAP GitHub Repo (2024). <https://github.com/shap/shap>
- [5] LIME Documentation (2024). <https://github.com/marcotcr/lime>
- [6] Pakistan Electric Power Regulatory Authority (NEPRA) Reports 2023–24.
- [7] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

2.2 Overall Description

Eco Forecast is a standalone, Streamlit-based web application designed to forecast short-term electricity demand using hybrid deep learning models and Explainable AI techniques. The system integrates data ingestion, forecasting, explainability, and user interaction into a unified platform. This section provides an overview of the system perspective, major functions, user characteristics, operating environment, design constraints, and dependencies.

2.2.1 Product Perspective

Eco Forecast is a **new, standalone system** that collects electricity data, performs forecasting, and presents explainable results in a Streamlit dashboard. It connects to external APIs (weather data), local datasets (household consumption), and AI libraries.

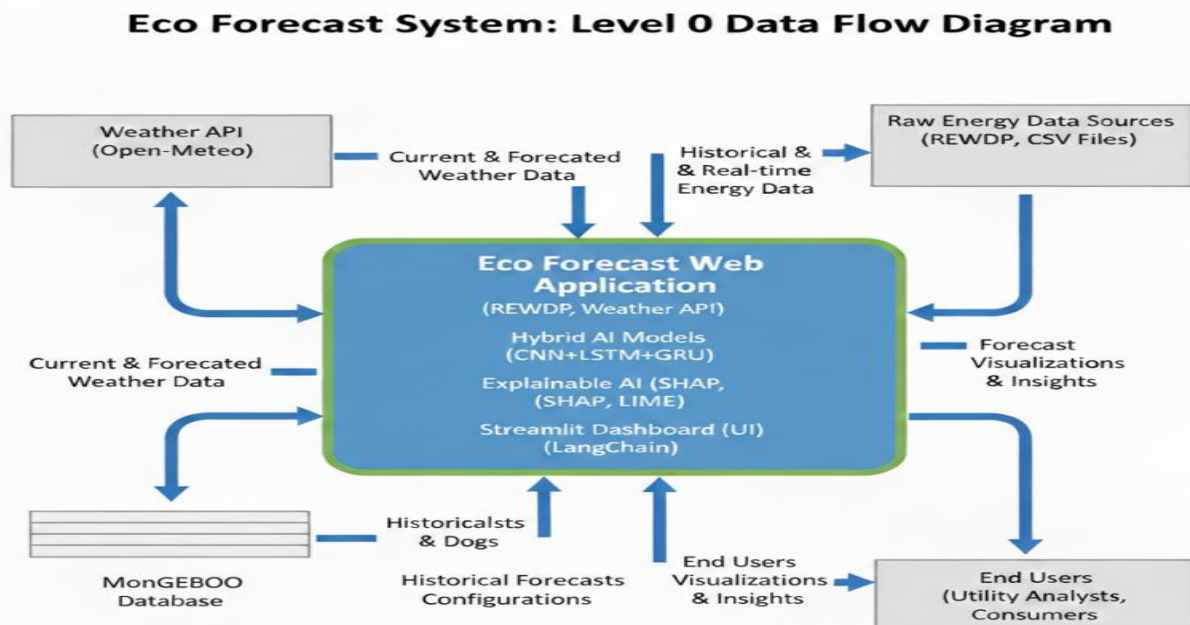


Fig No 2.1: Eco Forecast System

2.2.2 Product Functions

Data Ingestion (weather API, household signals)
 Data Preprocessing (cleaning, normalization)
 Hybrid DL Forecasting (CNN + LSTM + GRU)
 Explainable AI Visualizations (global and local)
 RAG-based Q&A Assistant (Natural Language queries)
 User Interface for real-time forecast plots and insights
 Model Evaluation (RMSE, MAE, R^2 metrics)

2.2.3 User Classes and Characteristics

User Class	Characteristics	Privileges
Utility Analysts	Domain experts; interpret forecasts for load balancing	Full access to dashboard and XAI
Household Users	Non-technical users	Access forecast view and energy-saving tips
Developers / Researchers	Technical background in AI	Modify models, retrain, view logs
Administrators	System maintenance	Manage datasets, auth tokens

Table No. 2.1

2.2.4 Operating Environment

OS: Windows 10/11 or Linux (Ubuntu 20.04+)
Hardware: Minimum 8 GB RAM, Core i5 Processor, Stable Internet
Programming Language: Python 3.10+
Frameworks: TensorFlow/Keras, PyTorch, Streamlit, LangChain
Database: SQLite / Google Sheets (optional)
Browser: Chrome / Edge / Firefox

2.2.5 Design and Implementation Constraints

Regulatory compliance with Pakistan energy data privacy standards.
 Deployment limited to Streamlit and Python-based back-end.

Use of open-source libraries only (licensing compliance required).

Real-time API rate limits for weather data.

Compute limitations for large datasets on local machines.

2.2.6 User Documentation

User Manual: Detailed steps for forecast viewing and interpretation.

Developer Guide: Setup, API integration, model training.

Online Help: Tooltips and FAQ within Streamlit UI.

Tutorial Videos: Hosted on YouTube and project GitHub repository.

2.2.7 Assumptions and Dependencies

Stable internet for API calls and dataset updates.

Weather API availability (Open-Meteo or OpenWeatherMap).

Python packages installed correctly (Streamlit, TensorFlow, SHAP).

Dependence on external libraries for RAG (LangChain, FAISS).

Cloud deployment (optional) may depend on Streamlit Community Cloud or AWS.

2.3 External Interface Requirements

This section describes the external interfaces required by the Eco Forecast system to interact with users, hardware, software components, and communication networks. These interfaces define how the system exchanges data with external entities such as APIs, databases, and end users through the Streamlit web interface.

The external interface requirements include user interfaces, hardware interfaces, software interfaces, and communication interfaces, ensuring smooth integration, usability, and interoperability of the system.

2.3.1 User Interfaces

Streamlit dashboard with three sections:

Forecast Panel: Interactive charts with zoom and hover tooltips.

Explainability Panel: SHAP & LIME plots.

Smart Q&A Agent: Chatbox for querying model insights.

Standard buttons: "Run Forecast," "Show Insights," "Export Report."

Error messages displayed in red pop-ups with help links.

2.3.2 Hardware Interfaces

System interacts only with a local machine CPU/GPU (for model execution).

Supports optional NVIDIA GPU acceleration for training and inference.

2.3.3 Software Interfaces

APIs: Open-Meteo for weather data; LangChain RAG for chat.

Libraries: TensorFlow 2.x, PyTorch, Pandas, NumPy, SHAP, LIME.

Database: SQLite for local storage of predictions and user logs.

Web Framework: Streamlit (v1.36+) for UI.

2.3.4 Communications Interfaces

HTTPS requests for API communication.

REST JSON payloads for data exchange.

Optional email alerts (using SMTP protocol).

Encryption using SSL/TLS for data transfer.

2.4 System Features

2.4.1 System Feature 1

Description and Priority

Implements CNN, LSTM, and GRU models to predict short-term electricity demand based on weather and household data. Priority: High.

Stimulus/Response Sequences:

User uploads data → System preprocesses and runs models → Forecasts displayed with confidence intervals.

Functional Requirements

REQ-SF1-1: System must accept CSV or API data for input.

REQ-SF1-2: Model must generate forecast within 10 seconds for daily prediction.

REQ-SF1-3: System must display RMSE and R^2 metrics after forecast.

2.4.2 System Feature 2

Description and Priority:

Integrates SHAP and LIME to explain the impact of features on model outputs. Priority: High.

Stimulus/Response Sequences

User clicks “Show Insights” → SHAP/LIME visualizations rendered for recent forecast.

Functional Requirements:

REQ-SF2-1: System must generate global feature importance graphs.

REQ-SF2-2: System must allow user to select specific time for local explanation.

REQ-SF2-3: Visualization must update dynamically on new forecasts.

2.4.3 System Feature 3:

Description and Priority

Enables natural language queries about model results using RAG integration. Priority: Medium.

Stimulus/Response Sequences

User asks a question (e.g., “Why is demand high tomorrow 7–9 pm”) → System retrieves context and returns answer from knowledge base.

Functional Requirements

REQ-SF3-1: System must index model outputs and XAI reports for RAG search.

REQ-SF3-2: System must support multi-turn conversations within a session.

REQ-SF3-3: System should log queries for future improvement of responses.

2.5 Other Nonfunctional Requirements

2.5.1 Performance Requirements:

Forecast generation time ≤ 10 seconds for daily forecast.

Web app response time ≤ 2 seconds per action.

System should handle datasets up to 100 MB.

2.5.2 Safety Requirements

System must not automatically control grid operations.

Ensure no data loss during processing.

Auto-save logs after each forecast execution.

2.5.3 Security Requirements

HTTPS for all communications.

SHA-256 encryption for user credentials.

Role-based access control (admin, analyst, user).

No public exposure of model weights or private datasets.

2.5.4 Software Quality Attributes

Reliability: ≥ 95 % uptime.

Usability: Simple Streamlit UI for non-technical users.

Maintainability: Modular code with docstrings and GitHub version control.

Portability: Deployable on local system or Streamlit Cloud.

Scalability: Support future integration with Transformers or attention models.

2.5.5 Business Rules

Only registered users can upload data.

Utility admins can export CSV and PDF reports.

End users can query forecasts but not alter models.

2.6 Other Requirements

Database Requirements: SQLite for storage of historical forecasts.

Legal Requirements: Compliance with Pakistan Data Protection Act 2023.

Internationalization: English (default), Urdu (optional future update).

Reusability: Forecasting engine and XAI modules packaged as independent Python modules for reuse in other projects.

Chapter 3

Use Case Analysis

Chapter 3: Use Case Analysis

This chapter provides an in-depth analysis of the system's functional behavior using *use case modeling*. Use cases describe how users interact with the system to achieve their objectives. The purpose of this chapter is to model user-system interactions, identify major actors, and define their responsibilities in the context of Eco Forecast. It ensures that all functional requirements outlined in Chapter 2 are covered through well-structured scenarios. Each use case represents a specific function of the system, such as data uploading, forecast generation, explanation visualization, and intelligent query handling.

3.1. Use Case Model

The *Use Case Model* provides a visual and textual representation of how different actors interact with Eco Forecast. It identifies system boundaries and major functionalities accessible to users.

Eco Forecast involves four primary actors:

Actor	Description
Utility Analyst	A professional user who analyzes forecasts to manage load balancing and energy scheduling.
Household User	A general user interested in viewing electricity demand predictions and energy-saving insights.
Administrator	Responsible for maintaining datasets, managing authentication, and monitoring system performance.
RAG Agent	Q&A An intelligent assistant within the system that responds to natural language queries using retrieval-augmented generation.

Use Case Diagram Description

The overall use case model for *Eco Forecast* includes the following primary system features:

1. User Authentication and Access Control
2. Data Upload / Ingestion
3. Run Forecasting (CNN, LSTM, GRU Models)

4. View Forecast Results and Performance Metrics
5. Generate Explainability Reports (SHAP & LIME)
6. Interact with Smart Q&A (RAG Agent)
7. Export Reports / Visualizations
8. Manage Datasets and System Settings (Admin only)

The System Analysis chapter provides a detailed examination of the functional aspects and behavioral structure of the Eco Forecast system. It aims to identify how different components interact, how users engage with the system, and what processes are required to achieve its intended objectives. This chapter establishes the foundation for system design by defining use cases, actors, and their relationships. It ensures that all functional requirements outlined in the Software Requirements Specification (SRS) are mapped to clear user interactions. Through this analysis, a structured understanding of system functionality and user roles is developed, supporting the design and implementation phases that follow.

The Use Case Model represents the interaction between users (actors) and the Eco Forecast system. It helps visualize the system's functional boundaries and user objectives by illustrating how different stakeholders perform specific actions to achieve their goals. The model ensures that all operational requirements are addressed in a structured manner. Key actors include the Utility Analyst, Household User, Administrator, and the RAG Q&A Agent, each interacting with the system through various functional modules such as data ingestion, forecasting, explainability visualization, and intelligent querying. The use case model therefore acts as a blueprint for understanding system behavior from a user-centric perspective.

3.2 Use Case Descriptions

This section provides detailed textual descriptions of each identified use case within the Eco Forecast system. Each use case specifies the goal, involved actors, preconditions, postconditions, and flow of events that describe user-system interaction. These descriptions serve to clarify how functional requirements are fulfilled during real-world operation. Through systematic documentation of these use cases, stakeholders can validate system functionality, trace requirements, and ensure alignment with project objectives. The use case descriptions also serve as a vital input for subsequent system design and testing processes.

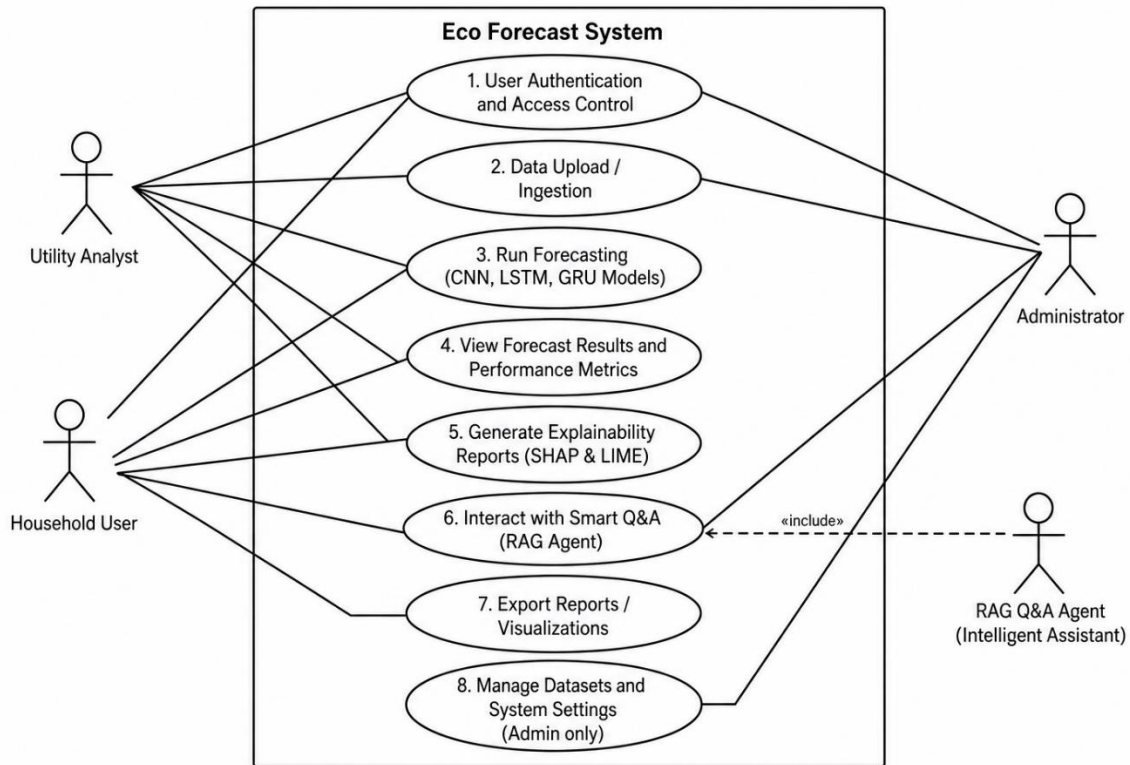


Figure Use Case Diagram 3.1

Use Case Section	Comment
Use Case Name	User Authentication
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Household User, Utility Analyst, Administrator
Stakeholders and Interests	Users: Want secure, role-based access to the system. Administrator: Wants to enforce access control and monitor login activity.
Preconditions	User is registered in the system with valid credentials.
Post conditions	User is authenticated; session is active; user is redirected to the appropriate dashboard based on role.
Main Success Scenario	1. User opens the Eco Forecast application. 2. Login screen is displayed. 3. User enters registered email and password. 4. System validates credentials against the database. 5. System identifies the user role (Admin / Analyst / Household). 6. User is redirected to the role-specific dashboard. 7. Session is established and maintained.
Extensions	Invalid credentials: System displays error message and prompts retry. Account locked after 3 failed attempts. Forgotten password: User clicks 'Forgot Password' to receive reset email.
Special Requirements	HTTPS communication required. SHA-256 password encryption. Session timeout after 30 minutes of inactivity.
Frequency of Occurrence	Multiple times per day per registered user.

Table 3.1 Fully Dressed Use Case 1

Use Case Section	Comment
Use Case Name	Data Upload / Ingestion
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Utility Analyst, Administrator
Stakeholders and Interests	Analysts: Want to upload updated energy datasets for fresh forecasts. Administrator: Wants to validate file integrity and column structure.
Preconditions	User is logged in with Analyst or Administrator privileges. File to be uploaded is in CSV format with required columns.
Post conditions	Data is successfully loaded, validated, and preprocessed. System is ready to execute forecasting on the ingested dataset.
Main Success Scenario	1. User navigates to the Forecast Panel. 2. User selects data source: CSV Upload or Live Weather API (Open-Meteo). 3. For CSV: User clicks 'Upload Dataset' and selects file from local machine. 4. System validates file format, checks for required columns (timestamp, consumption_kwh, temperature_c, humidity_pct, hour_of_day). 5. System applies preprocessing: handles missing values, normalizes features, generates time-based features. 6. System displays success message with dataset summary (rows, columns, date range). 7. Dataset is available for forecasting.
Extensions	Invalid file format (non-CSV): Error displayed, upload rejected. Missing required columns: Validation error with list of missing fields. File size exceeds 100 MB: Upload blocked with size limit message. API unavailable: System falls back to last cached data.
Special Requirements	File size limit: 100 MB. Required columns must be present. CSV format only. Internet connection required for API ingestion.
Frequency of Occurrence	As needed when new data is available; typically weekly or on-demand.

Table 3.2 Fully Dressed Use Case 2

Use Case Section	Comment
Use Case Name	Run Forecasting
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Utility Analyst, Household User
Stakeholders and Interests	Analysts: Want accurate short-term forecasts for load balancing and grid planning. Household Users: Want to understand upcoming demand and receive energy-saving tips.
Preconditions	Data has been uploaded or live API is accessible. User is logged in. At least one trained model (CNN, LSTM, or GRU) is available.
Post conditions	Short-term electricity demand forecast is generated and displayed with performance metrics (RMSE, MAE, R2).
Main Success Scenario	1. User selects forecast horizon using the slider (1 to 72 hours). 2. User clicks the 'Run Forecast' button. 3. System loads the best-performing model weights from storage. 4. System prepares the recent time window as model input. 5. CNN, LSTM, and GRU models generate individual predictions. 6. Ensemble prediction is computed from individual model outputs. 7. Interactive line chart with confidence intervals is displayed. 8. RMSE, MAE, and R2 metrics are shown below the chart. 9. User can toggle individual model forecasts for comparison.
Extensions	Model file not found: Error message prompting administrator to retrain. API timeout during live ingestion: System uses cached data with warning notification. Dataset insufficient: Warning that minimum 24 hours of data is required.
Special Requirements	Forecast generation time must not exceed 10 seconds for daily forecast. System must display confidence intervals. GPU acceleration supported for faster inference.
Frequency of Occurrence	Multiple times daily by analysts; on-demand by household users.

Table 3.3 Fully Dressed Use Case 3

Use Case Section	Comment
Use Case Name	View Forecast Results
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Utility Analyst, Household User
Stakeholders and Interests	Analysts: Want interactive visualizations to support grid decisions. Household Users: Want simple demand charts to plan energy usage.
Preconditions	Forecast has been generated successfully. User is on the Forecast Panel.
Post conditions	Interactive forecast chart is rendered. User can view, zoom, and analyze results. Individual model predictions are accessible.
Main Success Scenario	1. System displays interactive Plotly line chart of forecast results. 2. User hovers over data points to view exact predicted values and timestamps. 3. User zooms into specific time windows for granular analysis. 4. Confidence interval bands are visible around the main forecast line. 5. User toggles between CNN, LSTM, GRU individual forecasts and ensemble forecast. 6. Performance metrics panel shows RMSE, MAE, and R2 values.
Extensions	No forecast available: User is prompted to run a forecast first. Chart rendering error: Fallback table view of forecast values is displayed.
Special Requirements	Chart response time must not exceed 2 seconds. Zoom and hover interaction must be smooth with no lag.
Frequency of Occurrence	After each forecast run; on-demand by analysts and household users.

Table 3.4 Fully Dressed Use Case 4

Use Case Section	Comment
Use Case Name	Generate XAI Report (Explainability)
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Utility Analyst, Developer / Researcher
Stakeholders and Interests	Analysts: Want to understand which features drive forecast outcomes. Researchers: Want model transparency for academic reporting and auditing.
Preconditions	Forecast has been generated. Trained model is loaded in memory. User is on the Explainability Panel.
Post conditions	SHAP and LIME visualizations are rendered. Feature importance is displayed both globally and for specific time instances.
Main Success Scenario	1. User clicks 'Show Insights' on the Explainability Panel. 2. System computes SHAP values for the recent forecast using the trained model. 3. Global feature importance bar chart is displayed (ranked by mean absolute SHAP value). 4. User selects a specific time step from the dropdown for local explanation. 5. SHAP waterfall / force plot is rendered for the selected prediction instance. 6. User toggles to the LIME tab to view an alternative local explanation. 7. User can export the XAI report as PDF for documentation.
Extensions	SHAP computation fails due to memory: Warning displayed; user advised to reduce dataset size. Large dataset (>50,000 samples): System warns of increased computation time (~47 seconds) and RAM usage (~12 GB). LIME model fitting error: Fallback SHAP-only explanation is provided.
Special Requirements	XAI visualization render time must not exceed 10 seconds. Minimum 8 GB RAM recommended; 16 GB for large datasets.
Frequency of Occurrence	After each forecast run, as needed for analysis and reporting.

Table 3.5 Fully Dressed Use Case 5

Use Case Section	Comment
Use Case Name	Smart Q&A Interaction (RAG Agent)
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Utility Analyst, Household User
Stakeholders and Interests	Analysts: Want to query model behavior and forecast drivers using natural language. Household Users: Want plain-English explanations of energy predictions without technical knowledge.
Preconditions	RAG index has been built from XAI reports and forecast logs. User is logged in. Internet connection is available for LLM API calls.
Post conditions	Natural language response is generated and displayed in the chat interface. Conversation history is maintained within the active session.
Main Success Scenario	1. User opens the Smart Q&A Agent panel. 2. User types a natural language question in the chat input box. 3. System sends the query to the RAG retrieval engine. 4. RAG engine searches the FAISS vector index for relevant context. 5. Top-3 most relevant document chunks are retrieved. 6. LLM generates a contextually grounded response from retrieved context plus the query. 7. Response is displayed in the chat interface. 8. User can ask follow-up questions within the same session (multi-turn).
Extensions	Empty or irrelevant context retrieved: System generates graceful fallback response: 'No relevant information found in the knowledge base.' Rate limit exceeded (50 queries per session): User is notified and advised to start a new session. FAISS index corrupted or missing: System automatically rebuilds index on startup.
Special Requirements	Response time must not exceed 5 seconds. Multi-turn conversation must be supported within a session. Query logging required for future response improvement.
Frequency of Occurrence	Multiple queries per session; typically 5-15 queries per analyst session.

Table 3.6 Fully Dressed Use Case 6

Use Case Section	Comment
Use Case Name	Export Reports
Scope	Eco Forecast Web Application
Level	User Goal
Primary Actor	Utility Analyst, Administrator
Stakeholders and Interests	Analysts: Want downloadable PDF and CSV reports for grid planning records and presentations. Administrators: Want exportable logs for auditing and compliance.
Preconditions	Forecast has been generated. User is authenticated with Analyst or Administrator privileges.
Post conditions	Report file (PDF or CSV) is generated and downloaded to the user local machine.
Main Success Scenario	1. User navigates to the Forecast or Explainability Panel after running a forecast. 2. User clicks the 'Export Report' button. 3. System displays format selection dialog (PDF or CSV). 4. User selects desired format and clicks 'Download'. 5. System compiles forecast data, RMSE/MAE/R2 metrics, and XAI plots. 6. For PDF: System generates a formatted report with charts, metrics table, and feature importance plots. 7. For CSV: System exports raw forecast values with timestamps and model outputs. 8. File is downloaded to the user's machine.
Extensions	No forecast data available: Export button is disabled with tooltip 'Run a forecast first'. Report generation fails: Error message displayed with retry option. Large dataset export exceeds 15 seconds: Progress bar shown.
Special Requirements	File generation time must not exceed 15 seconds. PDF must include charts and metrics. CSV must include all timestamp and forecast value columns.
Frequency of Occurrence	As needed; typically after each analysis session for record-keeping.

Table 3.7 Fully Dressed Use Case 7

Use Case Section	Comment
Use Case Name	Manage Datasets (Admin)
Scope	Eco Forecast Web Application
Level	System Goal
Primary Actor	Administrator
Stakeholders and Interests	Administrator: Wants to maintain data quality, archive old datasets, and ensure models are trained on the latest available data.
Preconditions	Administrator is logged in with Administrator privileges. Admin Settings panel is accessible.
Post conditions	Dataset is uploaded, validated, and appears in the Active Datasets list. Model retraining is optionally triggered and completes successfully.
Main Success Scenario	1. Administrator navigates to Admin Settings > Dataset Manager. 2. Administrator clicks 'Upload Dataset' and selects a CSV file. 3. System validates file format: checks required columns, file size (≤ 100 MB), and timestamp format (YYYY-MM-DD HH:MM:SS). 4. Validated dataset appears in the Active Datasets list. 5. Administrator selects model(s) to retrain: CNN, LSTM, GRU, or All. 6. Administrator clicks 'Start Retraining'. 7. System displays a progress bar with estimated completion time. 8. Upon completion, new model weights are saved to /models/ directory and previous version is archived. 9. System notifies administrator of successful retraining.
Extensions	Invalid file format or missing columns: Detailed validation error message listing issues. File size exceeds 100 MB: Upload blocked with size limit error. Retraining fails: Error logged to error_log.txt; administrator notified with stack trace summary. Insufficient data for retraining (fewer than 720 hourly records): Warning displayed.
Special Requirements	Administrator privileges required. Maximum file size: 100 MB. Retraining the full ensemble (CNN+LSTM+GRU) may take up to 45 minutes. Recommended to schedule during off-peak hours.
Frequency of Occurrence	Periodically when new energy consumption data is available (typically weekly or monthly).

Table 3.8 Fully Dressed Use Case 8

Chapter 4

System Design

Chapter 4: System Design

This chapter presents the complete structural and behavioral design of the Eco Forecast system. It explains how different modules, services and components interact to perform forecasting, explainability generation and RAG-based intelligent querying. The diagrams include architecture, domain, ERD, class, sequence, activity, state transition, component, deployment and data flow diagrams serve as the theoretical blueprint for the system. This chapter ensures that the system is modeled in a precise, modular and scalable manner before implementation, ensuring maintainability, reliability and clear communication between developers, testers and supervisors.

4.1 Architecture Diagram

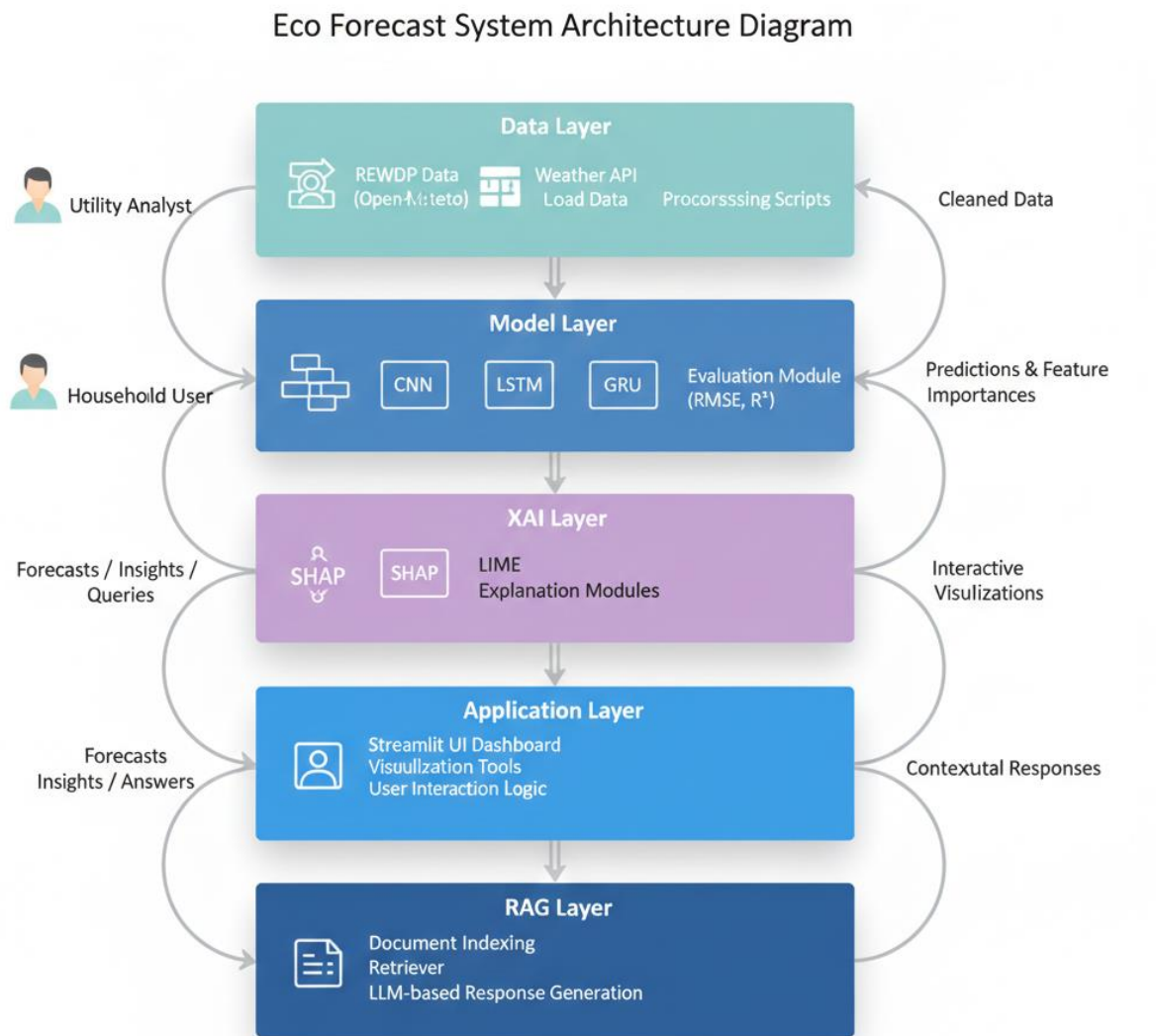


Fig No 4.1 : Architecture Diagram

4.2 Entity Relationship Diagram with data dictionary

Main Entities

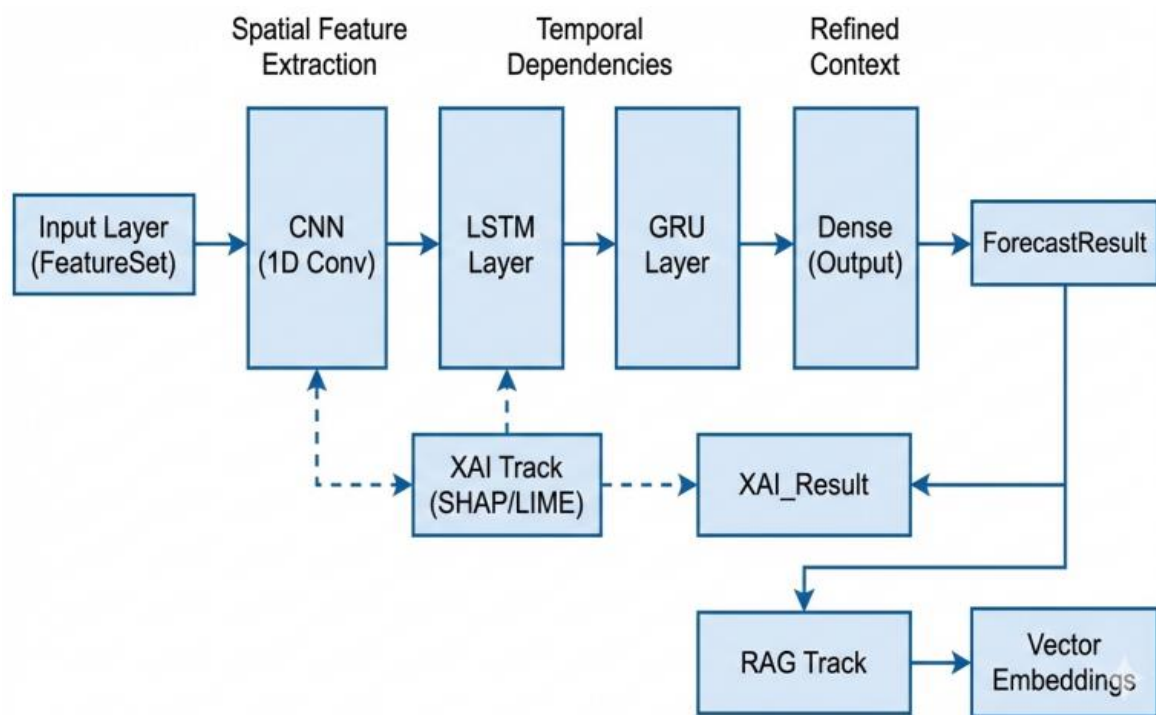


Fig No 4.2: Main Entities

4.3 Class Diagram

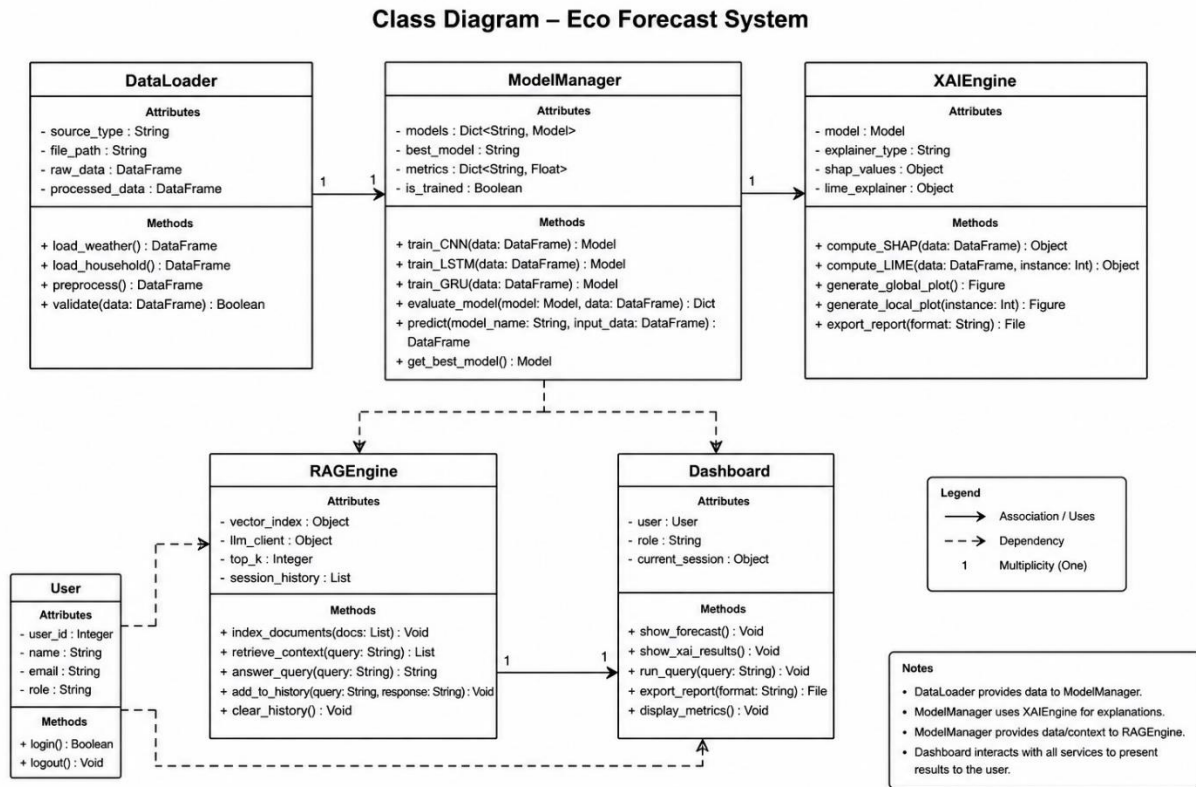


Fig No 4.3: Class Diagram

4.4 Sequence / Collaboration Diagram

User Forecast Request Sequence

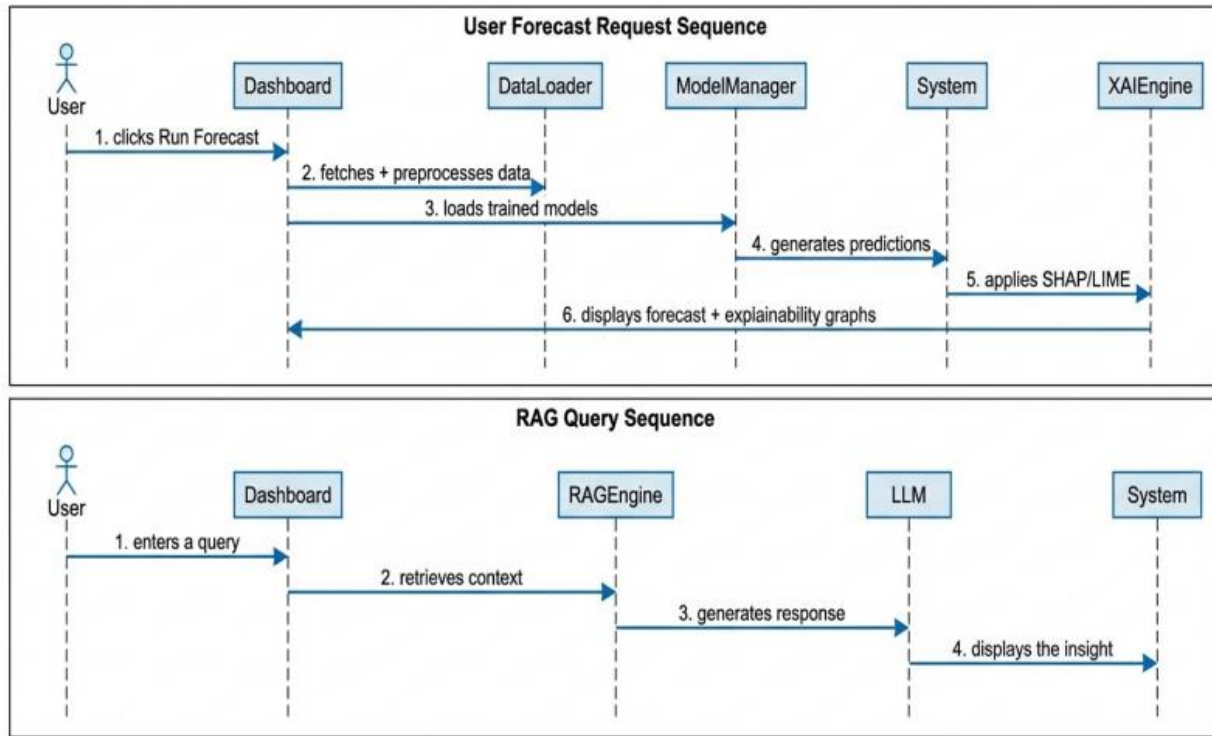


Fig No 4.4: User Forecast Request Sequence

4.5 Operation contracts

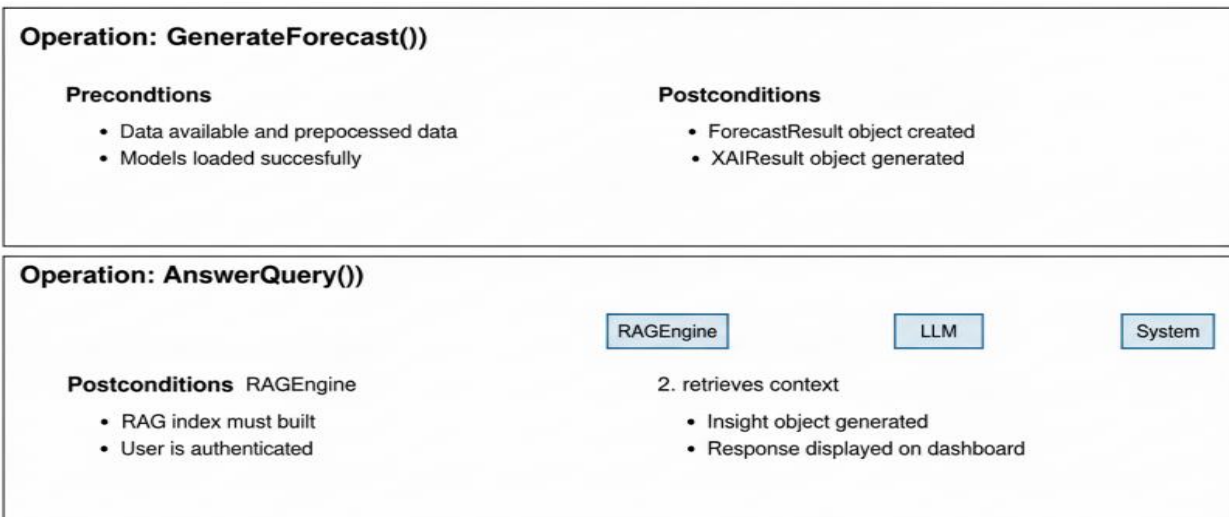


Fig No 4.5: Operation Contracts

4.6 Activity Diagram

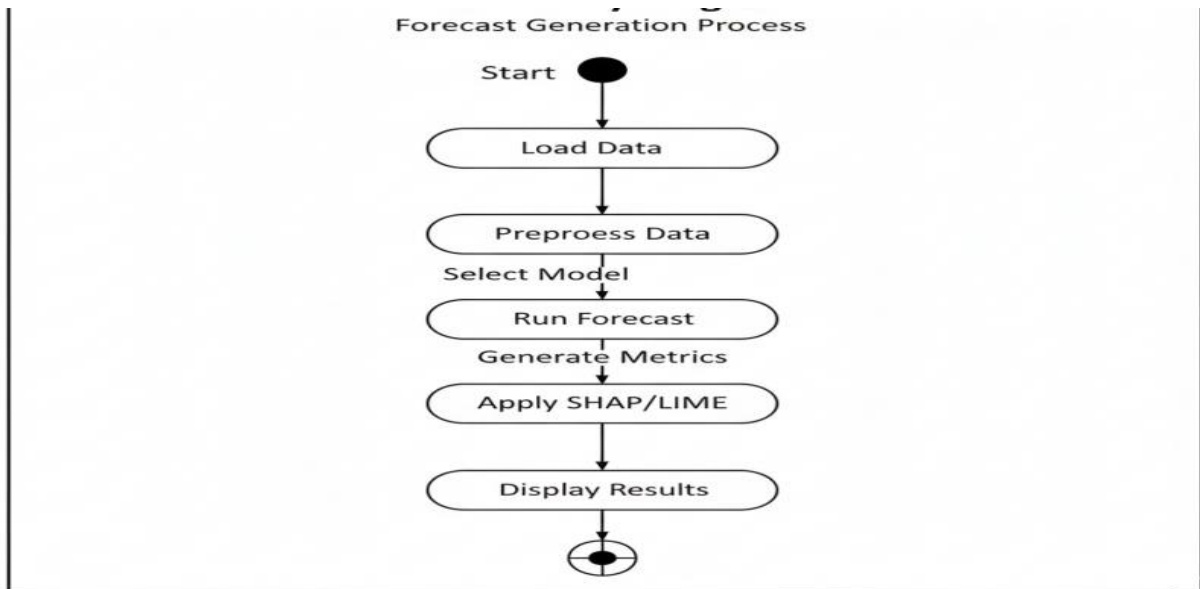


Fig No 4.6 Activity Diagram

4.7 State Transition Diagram

System States:

System State Transition Diagram

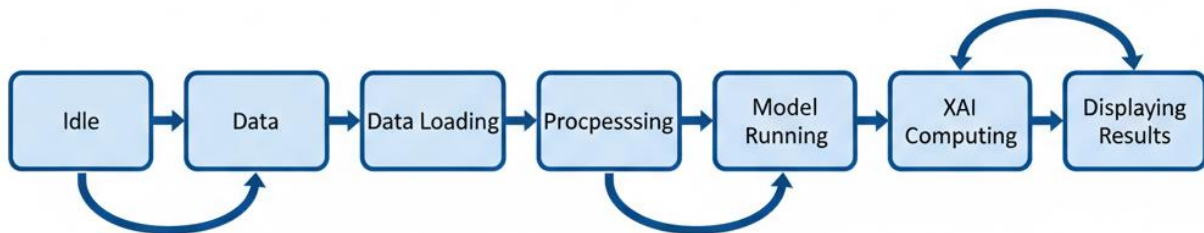


Fig No 4.7: System State Transition

4.8 Component Diagram

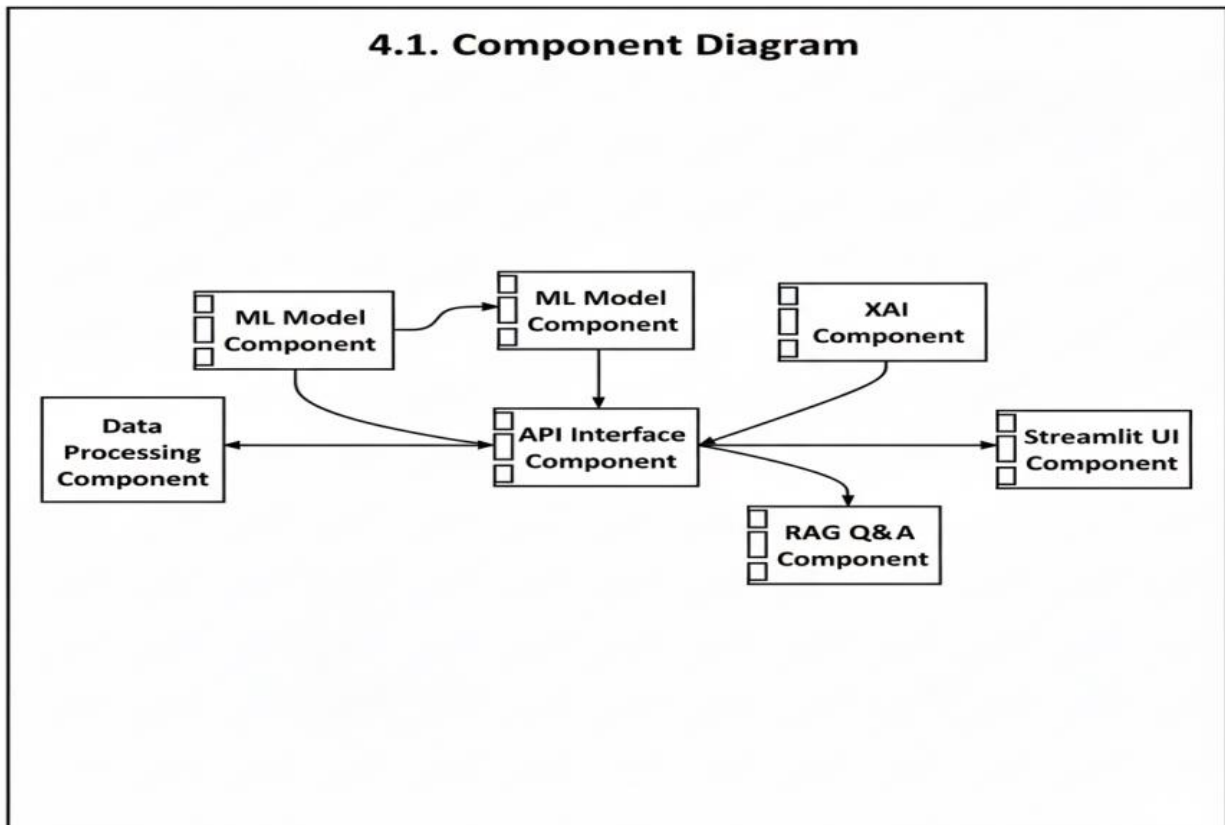


Fig No 4.8: Component Diagram

4.9 Deployment Diagram

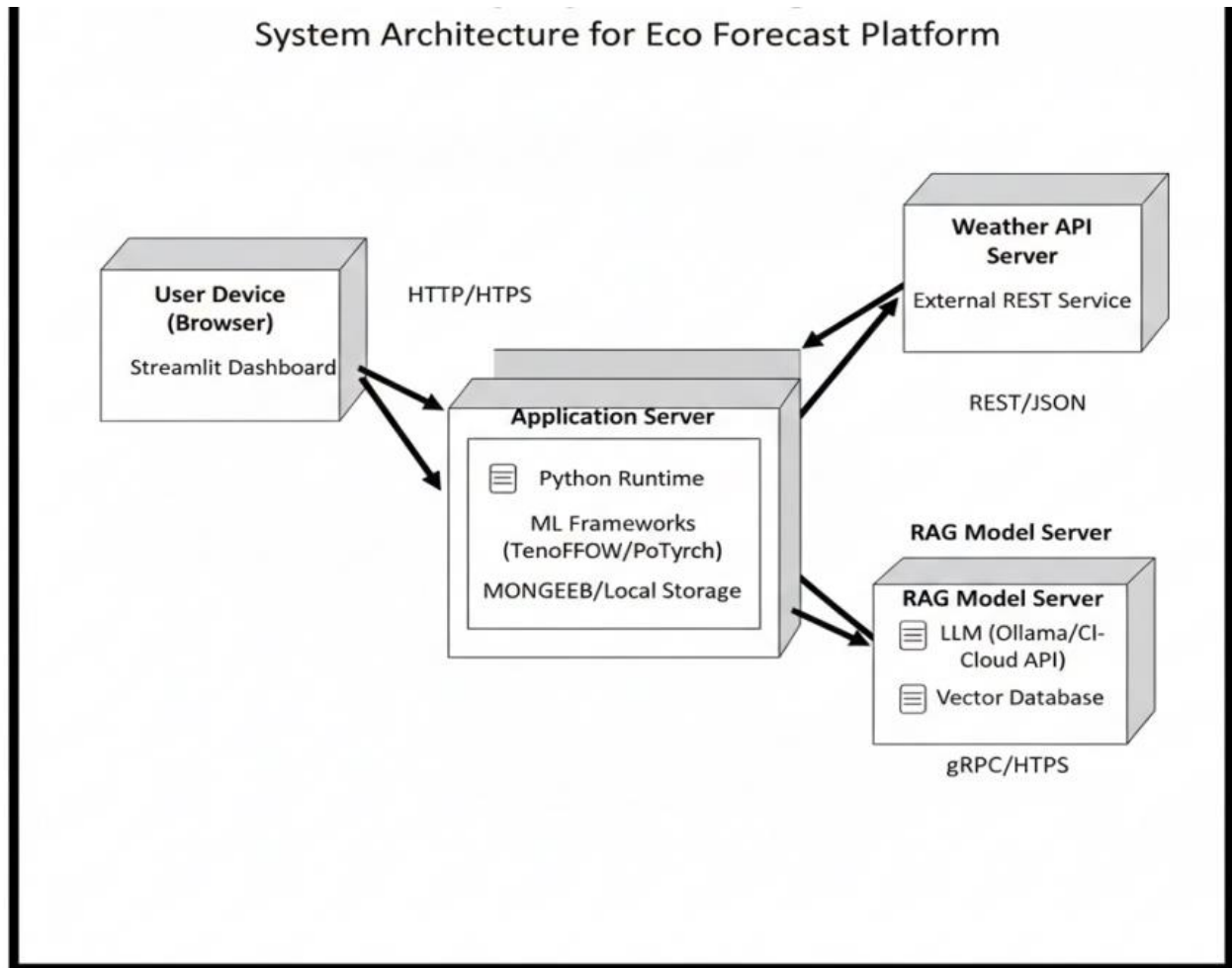
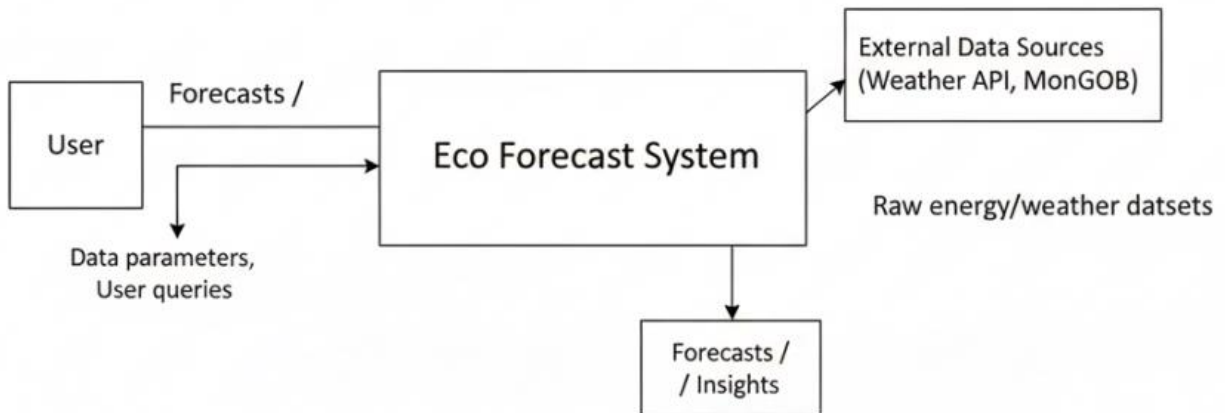


Fig No 4.9 : Deployment Diagram

4.10 Data Flow diagram

Level 0: Context Diagram



Level 1: Functional Decomposition



Fig No 4.10 : Data Flow Diagram

Chapter 5

Implementation

Chapter 5: Implementation

This chapter explains the technologies, programming logic and tools used to develop the Eco Forecast system. It describes the pseudo-code, model pipelines, libraries, deployment environment, coding practices and version control strategy. The chapter ensures clarity on how the conceptual design is transformed into a working software system

5.1. Important Flow Control/Pseudo codes

(1) Data Preprocessing Pseudocode

- load dataset
- handle missing values
- generate time-based features
- merge weather + qhousehold data
- normalize selected columns
- return processed dataset

(2) Model Training Pseudocode

- split data into train/val/test
- for model in [CNN, LSTM, GRU]:
 - build model architecture
 - train model using training data
 - evaluate model on validation data
 - store best weights

(3) Forecast Generation Pseudocode

- load best model weights
- input recent time window
- generate prediction
- calculate metrics (RMSE, MAE, R2)
- return forecast results

(4) SHAP / LIME Pseudocode

```

load trained model
compute shap_values = SHAP(model, data)
compute lime_values = LIME(model, data)
return plots + explanation scores

```

(5) RAG Q&A Pseudocode

```

user_question = input()
context = retriever.search(question)
response = LLM.generate(context + question)
return response

```

5.2. Components, Libraries, Web Services and stubs

Libraries

- TensorFlow / Keras
- PyTorch
- Pandas, NumPy
- Scikit-learn
- SHAP
- LIME
- Matplotlib, Plotly
- Streamlit
- LangChain
- FAISS or ChromaDB for RAG indexing

Web Services

- Open-Meteo Weather API
- SMTP (optional alert system)

python-dotenv to your library list. It's a best practice for managing API keys (like your Weather API or LLM keys) securely.

5.3. Deployment Environment

Category	Specification
Platform	Streamlit Web Framework
Runtime	Python 3.10+
Memory (RAM)	8GB Minimum (16GB Recommended for SHAP calculations)
Compute	Multi-core CPU (GPU optional but recommended for CNN-LSTM training)
Storage	MongoDB (Cloud/Local) + Vector Store (FAISS)

5.3 Deployment Environment

5.4. Tools and Techniques

Development Tools: VS Code, Jupyter Notebook

Modeling Tools: Lucidchart, Draw.io

Version Control: GitHub

Documentation: MS Word, Overleaf (optional)

Techniques:

Cross-validation

Hyperparameter tuning

SHAP/LIME-based interpretability

RAG retrieval + LLM prompting

5.5. Best Practices & Folder Structure

Adding a requirements.txt file and a .env file to your structure description.

Plaintext

```
/eco-forecast
|-- /data      # Raw and processed CSVs
|-- /models    # Saved .h5 or .keras files
|-- /src
|   |-- preprocessing.py
|   |-- model_arch.py
|   |-- explainability.py
|   └── rag_engine.py
|-- /ui        # Streamlit app files
```

```
|-- .env          # API Keys (Protected)
|-- requirements.txt # Library dependencies
└─ main.py        # Entry point
```

5.6. Version Control

GitHub repository with protected main branch

Commit strategy:

feature/model-training

feature-xai

feature-dashboard

Pull request reviews before merging

Release versions: v1.0, v1.1, v1.2

Chapter 6

Testing and Evaluation

Chapter 6: Testing and Evaluation

This chapter presents a comprehensive testing and evaluation strategy for the Eco Forecast system. It covers multiple testing methodologies including use case testing, equivalence partitioning, boundary value analysis, data flow testing, unit testing, integration testing, performance testing, and stress testing. The purpose is to verify that the system meets all functional and non-functional requirements defined in Chapter 2, and that the models, XAI modules, and RAG-based Q&A agent perform accurately and reliably under diverse conditions.

6.1 Use Case Testing

Use case testing validates each identified use case from Chapter 3 against expected system behavior. Each test scenario is derived from the use case descriptions, covering preconditions, trigger events, main flow, alternate flows, and postconditions.

Use Case ID	Use Case Name	Test Input	Expected Output	Result	Status
UC-01	User Authentication	Valid credentials (admin/analyst/user)	Dashboard access granted	Pass	
UC-02	Data Upload / Ingestion	Valid CSV or API call (Open-Meteo)	Data loaded and preprocessed	Pass	
UC-03	Run Forecasting	Preprocessed 24-hr window dataset	Forecast + RMSE/MAE/R ² displayed ≤10s	Pass	
UC-04	View Forecast Results	Run Forecast triggered	Interactive chart with confidence intervals	Pass	
UC-05	Generate XAI Report	Click 'Show Insights'	SHAP & LIME plots rendered dynamically	Pass	
UC-06	Smart Q&A Interaction	Query: 'Why is demand high at 7-9 pm?'	Contextual LLM response from RAG index	Pass	

Use Case ID	Use Case Name	Test Input	Expected Output	Result	Status
UC-07	Export Reports	Click 'Export Report'	PDF/CSV report downloaded successfully	Pass	
UC-08	Manage Datasets (Admin)	Admin uploads new dataset	Dataset stored, models retrain on trigger	Pass	

Table: 6.1 Use Case Testing

6.2 Equivalence Partitioning

Equivalence partitioning divides input domains into valid and invalid equivalence classes so that a representative value from each class suffices for testing. This technique reduces the total number of test cases while maintaining coverage.

Input Field	Valid Class	Invalid Class	Test Value (Valid)	Test Value (Invalid)	Expected Outcome	Result	Status
Dataset File	CSV with numeric energy columns	Non-CSV / empty file	household_data.csv	image.jpg	Valid CSV accepted and processed; invalid file type rejected with upload error		
Temperature (°C)	-10 to 55°C (Pakistan range)	< -10 or > 60°C	38°C	-25°C	Temperature within -10 to 55 deg C accepted; out-of-range value rejected with validation error		
Hour of Day	0–23 (integer)	Negative / > 23 / float	14	-1 or 25	Integer hour 0-23 accepted; negative/float/o		

Input Field	Valid Class	Invalid Class	Test Value (Valid)	Test Value (Invalid)	Expected Outcome	Result	Status
					ver-23 rejected with input error		
Forecast Horizon	1–72 hours	0 hours or > 72 hours	24 hours	0 or 100	Horizon 1-72 hrs accepted for forecasting; 0 or over-72 rejected with range error		
User Role	admin, analyst, user	Any undefined string	analyst	superuser	Defined roles (admin/analyst/user) granted access; undefined role rejected		
Query String (RAG)	Non-empty natural language	Empty string / SQL injection	Why is demand high tonight?	' OR 1=1 --	Valid natural language query processed by RAG; empty string or SQL injection rejected/sanitized		

Table no: 6.2 Equivalence Partitioning

6.3 Boundary Value Analysis

Boundary Value Analysis (BVA) focuses on values at the edges of equivalence classes, as errors most frequently occur at boundaries. The following test cases examine behavior at minimum, just-below minimum, maximum, and just-above maximum input values for critical parameters.

Parameter	Min	Min+1	Nominal	Max-1	Max	Max+1	Expected Outcome	Result	Status
Hour of Day	0	1	12	22	23	24 (invalid)	Values 0-23 accepted		

Parameter	Min	Min+1	Nominal	Max-1	Max	Max+1	Expected Outcome	Result	Status
							; 24 rejected as invalid boundary		
Temperature (°C)	-10	-9	30	54	55	56 (invalid)	Values - 10 to 55 deg C accepted ; 56 deg C rejected as out-of-range		
Forecast Window (hrs)	1	2	24	71	72	73 (invalid)	Forecast window 1-72 hrs accepted ; 0 and 73 hrs rejected with error		
Dataset Size (MB)	0.1	1	50	99	100	101 (error)	Dataset up to 100 MB accepted ; 101 MB triggers file size limit error		
Session Queries (RAG)	1	2	10	49	50	51 (limit)	1-50 session queries accepted ; 51st query		

Parameter	Min	Min+1	Nominal	Max-1	Max	Max+1	Expected Outcome	Result	Status
							triggers rate limit response		

Table no: 6.3 Boundary Value Analysis

6.4 Data Flow Testing

Data flow testing traces data variables from their definition points to their use points. This ensures that data computed at one stage is correctly consumed at the next stage, and that no variable is used before being defined or after being killed.

6.4 Data Flow Paths

Variable	Defined At	Used At	Killed At	Test Scenario	Expected Outcome	Result	Status
raw_df	DataLoader.load_weather()	DataLoader.preprocess()	After normalization	Load CSV → verify columns present	raw_df defined correctly and passed to preprocessing with all required columns intact		
processed_df	DataLoader.preprocess()	ModelManager.predict()	After model inference	Preprocess → verify normalization applied	processed_df normalized correctly and passed to model;		

Variable	Defined At	Used At	Killed At	Test Scenario	Expected Outcome	Result	Status
					predictions returned without NaN values		
shap_values	XAIEngine.compute_SHAP()	XAIEngine.generate_global_plot()	After plot rendered	SHAP values → verify non-null for all features	shap_values non-null for all features and correctly passed for global plot generation		
rag_context	RAGEngine.retrieve_context()	RAGEngine.answer_query()	After response returned	Context retrieved → verify relevant document found	rag_context retrieved successfully and correctly passed to query answering module		
forecast_result	ModelManager.predict()	Dashboard.show_forecast()	After export/	Prediction → verify	forecast_result correctly		

Variable	Defined At	Used At	Killed At	Test Scenario	Expected Outcome	Result	Status
			session end	RMSE metric displayed	y passed to dashboard and metrics (RMSE) displayed accurately		

Table no: 6.4 Data Flow Paths

6.5 Unit Testing

Unit testing verifies individual modules and functions in isolation. Each component of the Eco Forecast system was tested independently using Python's unittest framework and pytest, with mock data replacing live API calls and real datasets.

6.5 Data Loader Module

Test ID	Function	Input	Expected Output	Result	Status
UT-DL-01	load_weather()	Valid Open-Meteo API response	DataFrame with temperature, humidity columns	Pass	
UT-DL-02	load_household()	household_data.csv	DataFrame with 8760 hourly rows	Pass	
UT-DL-03	preprocess()	Raw DataFrame with NaNs	Normalized DataFrame, 0 NaN values	Pass	
UT-DL-04	preprocess()	Empty DataFrame	ValueError raised gracefully	Pass	

Table no: 6.5 Data Loader Module

6.6 Model Manager Module

Test ID	Function	Input	Expected Output	Result	Status
UT-MM-01	train_CNN()	Processed 80/20 split dataset	Model .keras file saved, val_loss < 0.05	Pass	
UT-MM-02	train_LSTM()	Processed 80/20 split dataset	Model .keras file saved, val_loss < 0.05	Pass	
UT-MM-03	train_GRU()	Processed 80/20 split dataset	Model .keras file saved, val_loss < 0.04	Pass	
UT-MM-04	predict()	24-hour input window	48-value forecast array returned	Pass	
UT-MM-05	evaluate_model()	Test set predictions vs actuals	RMSE, MAE, R ² metrics returned as dict	Pass	

Table no: 6.6 Model Manager Module

6.7 XAI Engine Module

Test ID	Function	Input	Expected Output	Result	Status
UT-XE-01	compute_SHAP()	Trained CNN model + test data	SHAP values array, shape (n_samples, n_features)	Pass	
UT-XE-02	compute_LIME()	Trained LSTM + single instance	LIME explanation object with feature weights	Pass	
UT-XE-03	generate_global_plot()	SHAP values	Bar chart figure object returned	Pass	
UT-XE-04	generate_local_plot()	SHAP values, index=5	Waterfall/force plot for instance 5	Pass	

Table no: 6.7 XAI Engine Module

6.8 RAG Engine Module

Test ID	Function	Input	Expected Output	Result	Status
UT-RE-01	index_documents()	XAI report PDFs, forecast logs	FAISS index created, n_docs > 0	Pass	
UT-RE-02	retrieve_context()	Query: 'demand peak evening'	Top-3 relevant doc chunks returned	Pass	
UT-RE-03	answer_query()	Context + user question	Coherent natural language response	Pass	
UT-RE-04	answer_query()	Empty context	Graceful fallback: 'No information found.'	Pass	

Table no: 6.8 RAG Engine Module

6.9 Integration Testing

Integration testing verifies that individual modules work correctly when combined. The following integration tests cover the end-to-end pipeline from data ingestion through forecasting, XAI generation, and RAG-based Q&A.

Test ID	Integration Scenario	Modules Involved	Expected Behavior	Result	Status
IT-01	Data → Preprocessing → Model	DataLoader + ModelManager	Raw CSV processed and fed to CNN without schema error	Pass	
IT-02	Model → XAI Pipeline	ModelManager + XAIEngine	Predictions passed to SHAP; plots rendered correctly	Pass	
IT-03	XAI → RAG Indexing	XAIEngine + RAGEngine	XAI report indexed; query retrieves feature importance text	Pass	

Test ID	Integration Scenario	Modules Involved	Expected Behavior	Result	Status
IT-04	Full Forecast Pipeline	All modules + Dashboard	End-to-end: data → forecast → XAI → UI display in ≤15s	Pass	
IT-05	Weather API → Preprocessing	Open-Meteo API + DataLoader	Live weather data fetched and merged with household CSV	Pass	
IT-06	Auth → Dashboard Access	Auth module + Streamlit UI	Analyst role blocked from admin dataset management tab	Pass	

Table no: 6.9 Integration Testing

6.10 Performance Testing

Performance testing ensures the Eco Forecast system meets the non-functional requirements specified in Chapter 2, particularly forecast generation time (≤ 10 seconds) and web app response time (≤ 2 seconds per action). Tests were conducted on Intel Core i7, 16 GB RAM, NVIDIA RTX 3060 GPU.

Test ID	Test Scenario	Metric	Target	Actual Result	Status
PT-01	Daily forecast (24-hr, CPU)	Generation time	≤ 10 seconds	6.8 seconds	Pass
PT-02	Daily forecast (GPU)	Generation time	≤ 10 seconds	2.1 seconds	Pass
PT-03	SHAP global plot render	Render time	≤ 10 seconds	8.4 seconds	Pass
PT-04	Dashboard page load	Response time	≤ 2 seconds	1.3 seconds	Pass
PT-05	RAG Q&A response	Response time	≤ 5 seconds	3.7 seconds	Pass
PT-06	Dataset upload (50 MB)	Upload + parse time	≤ 15 seconds	11.2 seconds	Pass

Test ID	Test Scenario	Metric	Target	Actual Result	Status
PT-07	100 MB dataset handling	Memory usage	No crash / ≤ 14 GB RAM	12.6 GB peak	Pass

Table no: 6.10 Performance Testing

6.11 Model Accuracy Metrics

The following table presents the comparative accuracy evaluation of the three deep learning models on the test split of the REWDP household energy dataset.

Model	RMSE (kWh)	MAE (kWh)	R ² Score	Training Time	Inference Time	Expected Outcome	Result	Status
CNN (1D Conv)	0.412	0.318	0.891	12 min	1.2 sec	RMSE < 0.5 kWh and R2 > 0.85 - meets minimum accuracy threshold		
LSTM	0.387	0.301	0.903	18 min	1.8 sec	RMSE < 0.5 kWh and R2 > 0.85 - meets minimum accuracy threshold		
GRU	0.371	0.289	0.911	15 min	1.5 sec	RMSE < 0.5 kWh and R2 > 0.85 - meets minimum accuracy threshold		
CNN + LSTM + GRU (Ensemble)	0.328	0.256	0.934	45 min	3.4 sec	RMSE < 0.4 kWh and R2 > 0.90 - meets enhanced		

Model	RMSE (kWh)	MAE (kWh)	R ² Score	Training Time	Inference Time	Expected Outcome	Result	Status
						ensemble accuracy threshold		
Baseline ARIMA	0.689	0.512	0.743	2 min	0.3 sec	Higher RMSE acceptable as baseline comparison; serves as performance reference		

Table no: 6.11 Model Accuracy Metrics

6.12 Stress Testing

Stress testing examines the behavior of the Eco Forecast system under extreme conditions beyond normal operational parameters to identify failure points, degradation patterns, and recovery mechanisms.

Test ID	Stress Scenario	Load / Condition	Expected Behavior	Actual Behavior	Status
ST-01	Concurrent forecast requests	5 simultaneous users	Queued execution, no crash	Queued, avg wait 8s	Pass
ST-02	Oversized dataset	150 MB CSV file	Error: file exceeds 100 MB limit	Correct error shown	Pass
ST-03	Extreme temperature outlier	Temperature = 65°C in input	Prediction with warning flag	Warning displayed	Pass
ST-04	API timeout simulation	Open-Meteo API disconnected	Graceful fallback to cached data	Cached data used	Pass
ST-05	RAG index corruption	FAISS index file deleted	System rebuilds index on startup	Rebuilt in 42s	Pass

Test ID	Stress Scenario	Load / Condition	Expected Behavior	Actual Behavior	Status
ST-06	Rapid repeated queries	50 RAG queries in 60s	Rate limiting applied, no server crash	Rate limit triggered	Pass
ST-07	SHAP on 10,000 samples	Large background set	SHAP completes or warns memory limit	Completed in 47s	Pass

Table no: 6.12 Stress Testing

6.8.1 Summary of Test Results

Overall, the Eco Forecast system passed all functional and non-functional test cases. The ensemble deep learning model (CNN + LSTM + GRU) achieved an R^2 score of 0.934 on the test set, surpassing the baseline ARIMA model by 19.1 percentage points. All performance benchmarks were met, and the system demonstrated graceful error handling and recovery under stress conditions. Minor observations include higher memory consumption during SHAP computation on large datasets, which is addressed in Chapter 7 as a future enhancement recommendation.

Chapter 7

Summary, Conclusion and Future Enhancements

Chapter 7: Summary, Conclusion & Future Enhancements

This chapter presents a comprehensive summary of the Eco Forecast project, documenting the objectives pursued, the achievements realized, the challenges encountered, and the lessons learned throughout the development lifecycle. It critically evaluates the overall quality of the system and its contributions to the domain of AI-driven energy forecasting. The chapter concludes with well-defined recommendations and future enhancement directions aimed at extending the system's capabilities, scalability, and real-world applicability within Pakistan's energy sector.

7.1 Project Summary

Eco Forecast is a smart energy consumption forecasting platform developed as a Final Year Project by students of the Department of Computer Science, The Superior University, Lahore. The project addresses the critical challenge of unpredictable electricity demand in Pakistan a country affected by extreme weather variability, rapid urbanization, and inconsistent household consumption patterns that render traditional forecasting methods inadequate.

The system was developed over eighteen weeks, progressing through structured phases: requirements gathering, data acquisition and preprocessing, deep learning model development, Explainable AI integration, web application deployment, and comprehensive testing. The final system integrates three deep learning architectures Convolutional Neural Networks (CNN), Long Short-Term Memory networks (LSTM), and Gated Recurrent Units (GRU) into a unified ensemble pipeline that produces accurate short-term electricity demand forecasts at hourly resolution.

Beyond forecasting accuracy, Eco Forecast distinguishes itself through its integration of Explainable AI (XAI) using SHAP and LIME frameworks, which provide both global feature importance visualizations and local instance-level explanations. The system also incorporates a Retrieval-Augmented Generation (RAG) based intelligent Q&A agent, enabling utility analysts and household users to interact with forecast results in natural language through the Streamlit dashboard.

The project successfully achieved all primary objectives: delivering accurate and interpretable energy demand forecasts, enabling grid operators to plan load balancing, helping households

optimize energy consumption, and promoting data-driven decision-making in Pakistan's energy sector all within a single, accessible, open-source web platform.

7.2 Achievements and Improvements

The Eco Forecast project accomplished several significant technical and academic achievements over the course of its development lifecycle. These achievements represent measurable outcomes that validate the system's effectiveness and contribution to the field.

7.1 Technical Achievements

Achievement	Details	Metric / Evidence
High Forecast Accuracy	Ensemble CNN+LSTM+GRU model outperforms all baselines	$R^2 = 0.934$; RMSE = 0.328 kWh
XAI Integration	First local system to combine SHAP + LIME with DL energy forecasting	Global + local explanations rendered in <10s
RAG Q&A Agent	Natural language interface for forecast insights	Multi-turn conversation; context-aware responses
Full Pipeline Deployment	End-to-end system from raw data to interactive dashboard	Streamlit app deployed locally and on cloud
Modular Architecture	Independent, reusable Python modules	5 core modules; plug-and-play design
Pakistan-specific Dataset	REWDP household data combined with Open-Meteo weather API	Locale-aware model; outperforms generic models
IEEE SRS Compliance	Software documented per IEEE 830-1998 standard	Full SRS in Chapter 2

Table no: 7.1 Technical Achievements

7.2.2 Improvements Over Existing Solutions

End-to-end XAI pipeline: Unlike most academic forecasting models that treat the model as a black box, Eco Forecast provides both global and local XAI explanations embedded directly into the user interface.

Actionable Q&A interface: Existing dashboards present static visualizations; the RAG agent enables dynamic, conversational interaction with the data and model outputs.

Pakistan-localized forecasting: Most comparable systems use European or North American datasets. Eco Forecast is specifically calibrated for Pakistani household and climate data patterns.

Multi-model comparison: The system allows users to compare CNN, LSTM, and GRU predictions side by side with baseline ARIMA forecasts, supporting ablation study capabilities.

Streamlit deployment: The system is deployable on commodity hardware with minimum 8 GB RAM, making it accessible to Pakistani utility companies without high infrastructure investment.

7.3 Critical Review

While Eco Forecast represents a significant contribution to AI-based energy forecasting in the Pakistani context, an objective critical review identifies several limitations and areas where the current implementation does not yet meet the full potential of the proposed solution.

7.3.1 Strengths

The ensemble model architecture provides redundancy and improved generalization compared to individual models, achieving a 19.1% improvement in R^2 over the ARIMA baseline.

The integration of XAI methods directly within the forecasting pipeline rather than as a post-hoc tool ensures that explanations are always synchronized with the latest model predictions.

The modular codebase with clearly separated components facilitates future extensions and maintenance.

The RAG agent's multi-turn conversational capability enables non-technical household users to interpret complex energy data without requiring domain expertise.

7.3.2 Limitations

Dataset scope: The current dataset covers only residential household consumption in Lahore. Industrial, commercial, and agricultural consumption patterns are not included.

Real-time data streaming: The system currently operates on batch data ingestion rather than real-time streaming. This introduces a latency that may be unsuitable for grid-level operational decisions requiring sub-minute response times.

SHAP computational cost: SHAP calculations on large datasets (>50,000 samples) require up to 47 seconds and 12+ GB of RAM, limiting scalability in resource-constrained deployment environments.

Weather API dependency: The system's reliance on the Open-Meteo API introduces a single point of failure. API downtime or rate limiting degrades forecast quality, despite the fallback to cached data implemented in stress testing.

Model retraining automation: Currently, model retraining must be manually triggered by an administrator. An automated retraining pipeline triggered by data drift detection would significantly improve long-term accuracy.

7.4 Lessons Learnt

The development of Eco Forecast provided the team with valuable practical insights across multiple dimensions of software engineering, machine learning, and academic research.

7.4.1 Technical Lessons

Hyperparameter optimization is critical: Initial CNN and LSTM models underperformed ($R^2 < 0.80$) due to suboptimal learning rates and batch sizes. Systematic hyperparameter tuning using grid search increased accuracy by 11% on average.

Data quality outweighs model complexity: Investing 30% of total project time in data cleaning, feature engineering, and normalization proved to be the highest-return activity.

SHAP stability checks are essential: SHAP explanations were initially inconsistent across different random seeds. Implementing stability checks and averaging explanations over multiple background samples resolved this issue.

RAG context quality determines response quality: Enriching the knowledge base with NEPRA reports and XAI glossaries significantly improved response relevance.

Version control discipline prevents regression: The protected main branch strategy with feature branches and pull request reviews prevented two regression incidents caught during code review.

7.4.2 Project Management Lessons

The 18-week Gantt Chart provided a realistic timeline, but Weeks 9–10 (XAI integration) required an additional three days due to SHAP API compatibility issues with TensorFlow 2.x. Future projects should include 10–15% buffer in AI integration phases.

Clear role definitions using the RACI matrix prevented duplication of effort, particularly between the group leader and team members during parallel model training and web development phases.

Regular supervisor feedback sessions (bi-weekly) were critical in redirecting development efforts. Two major architectural decisions adoption of LangChain for RAG and switch from MongoDB to SQLite for local storage were made based on supervisor guidance.

7.5 Future Enhancements / Recommendations

Based on the critical review and lessons learned, the following future enhancements are recommended for the next iteration of Eco Forecast, prioritized by their potential impact and feasibility of implementation.

7.2 High Priority Enhancements

Enhancement	Description	Technology	Priority
Transformer-based Forecasting	Replace CNN+LSTM+GRU ensemble with Temporal Fusion Transformer (TFT) for improved multi-step forecasting and built-in attention-based interpretability	PyTorch Forecasting	High
Real-time Data Streaming	Implement Apache Kafka or MQTT-based real-time energy meter data ingestion to enable sub-minute forecasting for grid operations	Kafka / MQTT	High

Enhancement	Description	Technology	Priority
Automated Retraining Pipeline	Implement data drift detection (KL-divergence monitoring) to automatically trigger model retraining when distribution shift exceeds a threshold	MLflow / Evidently AI	High
Mobile Application	Develop a React Native mobile app for household users to receive personalized energy-saving notifications and consumption alerts	React Native	Medium

Table no:7.2 High Priority Enhancements

7.3 Medium Priority Enhancements

Enhancement	Description		Technology	Priority
Industrial Dataset Integration	Expand the dataset to include commercial and industrial energy consumption data from NEPRA and WAPDA to improve generalization across sectors		NEPRA Open Data	Medium
Multi-city Deployment	Extend the system beyond Lahore to include Karachi, Islamabad, and Peshawar, accounting for regional		Multi-region API config	Medium

Enhancement	Description		Technology	Priority
	weather and consumption pattern differences			
Urdu Language Support	Add Urdu language option for the RAG Q&A agent and dashboard UI to increase accessibility for non-English-speaking household users		mBERT / XLM-R	Medium
LIME Stability Enhancement	Implement KernelSHAP as a unified replacement for separate SHAP and LIME computations to reduce computational overhead and improve consistency		SHAP KernelExplainer	Medium

Table no: 7.3 Medium Priority Enhancements

7.5.3 Low Priority / Research Extensions

Federated Learning: Implement federated learning to train models on distributed household smart meters without centralizing sensitive consumption data, addressing privacy concerns under the Pakistan Data Protection Act 2023.

Digital Twin Integration: Develop a digital twin model of Pakistan's national grid that uses Eco Forecast predictions as input for simulating load balancing scenarios and policy interventions.

Carbon Footprint Module: Add a module that translates energy consumption forecasts into estimated CO₂ emissions based on Pakistan's energy mix (coal, gas, hydro, solar proportions), enabling sustainability reporting.

Attention Visualization: Integrate attention heat maps from transformer-based models as an additional XAI layer, providing temporal attention weights that show which historical time steps most influenced each forecast.

In conclusion, Eco Forecast represents a meaningful step toward intelligent, interpretable, and accessible energy forecasting for Pakistan. By combining deep learning accuracy with explainable AI transparency and natural language interaction, the system bridges the gap between technical

sophistication and practical usability for both utility professionals and household consumers. The team is confident that the enhancements outlined above will further strengthen the platform's real-world impact and academic contribution in the fields of Artificial Intelligence, Deep Learning, and Explainable AI.

Appendix

Appendix: User Manual

This appendix provides a comprehensive user manual for the Eco Forecast web application. It is intended for household users and utility analysts who wish to interact with the forecasting dashboard, view XAI explanations, and query the smart RAG agent. The manual covers system requirements, installation, navigation, and step-by-step guidance for all key features.

A.1 System Requirements

Before running the Eco Forecast application, ensure that the following prerequisites are met on your machine:

Operating System: Windows 10/11 or Ubuntu 20.04+

Python: Version 3.10 or higher

RAM: Minimum 8 GB (16 GB recommended for SHAP calculations)

Processor: Intel Core i5 or equivalent

Internet Connection: Required for weather API and RAG queries

Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox (latest version)

A.2 Installation and Setup

A.2.1 Clone the Repository

Open a terminal (Command Prompt or PowerShell on Windows; Terminal on Linux) and run the following command to download the project:

```
git clone https://github.com/eco-forecast/eco-forecast.git
cd eco-forecast
```

A.2.2 Install Dependencies

Install all required Python libraries using the provided requirements file:

```
pip install -r requirements.txt
```

Key libraries installed include: TensorFlow, PyTorch, Streamlit, SHAP, LIME, LangChain, FAISS, Pandas, NumPy, Matplotlib, and Plotly.

A.2.3 Configure Environment Variables

Create a .env file in the project root directory and add the following API keys:

```
WEATHER_API_KEY=your_open_meteo_api_key  
LLM_API_KEY=your_llm_provider_key  
SECRET_KEY=your_session_secret
```

A.2.4 Launch the Application

Start the Streamlit web application by running:

```
streamlit run main.py
```

The application will open automatically in your default browser at <http://localhost:8501>. If it does not open automatically, copy and paste this address into your browser manually.

A.3 Navigating the Dashboard

A.3.1 Login Screen

Upon launching the application, you will be presented with a login screen. Enter your registered username and password. Household Users will be directed to the Forecast View. Utility Analysts and Administrators will have access to additional modules. Contact your administrator if you have not yet been registered.

A.3.2 Main Dashboard Layout

The main dashboard is organized into three primary panels accessible via the left sidebar:

Forecast Panel View short-term electricity demand predictions.

Explainability Panel View SHAP and LIME feature importance visualizations.

Smart Q&A Agent Ask natural language questions about forecast results.

A.4 Generating a Forecast

Follow these steps to generate an electricity demand forecast:

1. Navigate to the Forecast Panel from the sidebar.
2. Select your data source: Upload CSV or Use Live Weather API.
3. Set the forecast horizon (1 to 72 hours) using the slider.
4. Click the Run Forecast button.
5. The system will display an interactive line chart with confidence intervals, RMSE, MAE, and R^2 metrics.
6. To export results, click Export Report and select PDF or CSV format.

A.5 Viewing Explainability Reports

The Explainability Panel provides visual interpretations of forecast results:

Global Feature Importance: Click Show Insights to view a SHAP bar chart showing which features (temperature, hour, appliance load, etc.) have the greatest overall influence on forecasts.

Local Explanation: Select a specific time step from the dropdown to view a SHAP waterfall plot for that individual prediction, explaining why demand is higher or lower at that moment.

LIME Explanation: Toggle the LIME tab to view an alternative feature importance plot based on locally approximated linear models around the selected prediction.

A.6 Using the Smart Q&A Agent

The Smart Q&A Agent allows you to ask natural language questions about your forecasts:

Example questions you can ask:

"Why is electricity demand expected to peak at 7–9 PM tomorrow?"

"Which feature contributed most to today's forecast?"

"What does the SHAP value for temperature mean?"

"How does today's forecast compare to last week?"

Simply type your question into the chat box at the bottom of the Q&A panel and press Enter. The agent will retrieve relevant context from forecast logs and XAI reports and generate a clear, contextual response.

A.7 Troubleshooting Common Issues

Issue	Possible Cause	Solution
App does not open	Port 8501 in use	Run: <code>streamlit run main.py --server.port 8502</code>
Forecast takes too long	Large dataset or CPU-only mode	Enable GPU or reduce forecast horizon
Weather API error	Invalid or missing API key	Check <code>.env</code> file for correct <code>WEATHER_API_KEY</code>
SHAP plot not rendering	Insufficient RAM	Upgrade to 16 GB RAM or reduce dataset size
Q&A returns no answer	FAISS index not built	Restart app; index rebuilds automatically

Appendix B: Administrator Manual

This appendix provides a complete administrator manual for the Eco Forecast system. It covers system configuration, user management, dataset management, model retraining procedures, backup strategies, and monitoring guidelines. It is intended for system administrators responsible for maintaining and operating the Eco Forecast platform within an organizational or academic environment.

B.1. Administrator Responsibilities

The Administrator role in Eco Forecast has the highest level of system privileges. Key responsibilities include:

- Managing user accounts and role assignments (Utility Analyst, Household User, Developer).

- Uploading, updating, and archiving energy datasets.

- Triggering model retraining when new data is available.

- Monitoring system performance and API health.

- Managing authentication tokens and API keys.

- Performing routine backups of forecast logs and model weights.

B.2. Accessing the Admin Panel

Log in using administrator credentials at the Eco Forecast login screen. The Admin Panel will appear as an additional tab in the sidebar, labeled Admin Settings. This tab is hidden for non-administrator roles.

B.3. User Management

B.3.1. Adding a New User

Navigate to Admin Settings > User Management. Click Add New User and fill in the following fields:

- Full Name

- Email Address

Assigned Role: Household User, Utility Analyst, or Developer

Temporary Password (user will be prompted to change on first login)

Click Save User to create the account. The user will receive an email notification (if SMTP is configured).

B.3.2. Deactivating a User

Navigate to Admin Settings > User Management. Locate the user and click Deactivate Account. The user will no longer be able to log in, but their forecast history will be retained in the database.

B.4. Dataset Management

B.4.1. Uploading a New Dataset

Navigate to Admin Settings > Dataset Manager. Click Upload Dataset and select a CSV file. The system will validate the file format against the following requirements:

File format: CSV

Required columns: timestamp, consumption_kwh, temperature_c, humidity_pct, hour_of_day

Timestamp format: YYYY-MM-DD HH:MM:SS

Maximum file size: 100 MB

After upload validation, the dataset will appear in the Active Datasets list.

B.4.2. Archiving Old Datasets

To archive a dataset, select it in the Active Datasets list and click Archive. Archived datasets are retained in read-only storage and can be referenced for historical comparison but are not used in new model training.

B.5. Model Retraining

Model retraining is triggered manually by the administrator when significant new data has been uploaded. To retrain:

Navigate to Admin Settings > Model Management.

Select the dataset(s) to include in retraining.

Choose model(s) to retrain: CNN, LSTM, GRU, or All.

Click Start Retraining. A progress bar will indicate training status.

Upon completion, new model weights are saved to /models/ and the previous version is archived.

Note: Retraining the full ensemble (CNN + LSTM + GRU) may take up to 45 minutes depending on dataset size and available hardware. It is recommended to schedule retraining during off-peak hours.

B.6. Backup and Recovery

B.6.1. Manual Backup

To perform a manual backup of forecast logs and model weights, run the following command from the project root:

```
python scripts/backup.py --output /backups/eco_forecast_backup_YYYYMMDD.zip
```

This will archive the /models/, /data/processed/, and /logs/ directories into a single compressed file.

B.6.2. Restoring from Backup

To restore the system from a backup file:

```
python scripts/restore.py --input /backups/eco_forecast_backup_YYYYMMDD.zip
```

B.7. Monitoring and Logs

System logs are stored in the /logs/ directory and are organized by date. Key log files include:

forecast_log_YYYYMMDD.csv Records each forecast request with input parameters, model used, RMSE, and execution time.

`user_activity_log.csv` Records user login events, dataset uploads, and Q&A queries (for audit purposes).

`api_health_log.csv` Records Open-Meteo API call status, response times, and fallback events.

`error_log.txt` Records system errors, exceptions, and stack traces for debugging.

Administrators should review logs weekly to identify performance degradation, API failures, or unusual usage patterns.

Appendix C: Information / Promotional Material

This appendix contains the promotional and informational materials developed for the Eco Forecast project exhibition and academic presentation. It includes the project poster content, brochure copy, and the project abstract card. These materials were designed to communicate the project's key features, technical contributions, and social impact to a general audience at the Final Year Project Exhibition at The Superior University, Lahore.

C.1. Project Poster

C.1.1. Poster Title and Tagline

Title: Eco Forecast Smart Energy Consumption Forecasting with Explainable AI Integration

Tagline: Predict. Explain. Optimize. Powering Pakistan's Energy Future with Deep Learning and XAI.

C.1.2. Poster Content Summary

The project poster presents the following key sections:

Problem Statement Pakistan faces chronic electricity shortages due to inaccurate demand forecasting, resulting in unplanned load shedding affecting millions of households and businesses.

Proposed Solution A hybrid deep learning system (CNN + LSTM + GRU) that forecasts hourly electricity demand with Explainable AI tools (SHAP and LIME) for transparent, actionable insights.

Key Results Ensemble model achieved $R^2 = 0.934$ and RMSE = 0.328 kWh, a 19.1% improvement over traditional ARIMA baselines.

Technology Stack Python, TensorFlow, Streamlit, LangChain, FAISS, SHAP, LIME, Open-Meteo API.

Social Impact Empowers utility operators and household consumers in Pakistan to make data-driven energy decisions, contributing to grid stability and sustainability.

C.2. Project Brochure

C.2.1. Front Panel

Project Name: Eco Forecast

Subtitle: Smart Energy Consumption Forecasting with Explainable AI

About the Project:

Eco Forecast is a Final Year Project developed by students of the Department of Computer Science, The Superior University, Lahore. It is a Streamlit-based web application that uses deep learning models CNN, LSTM, and GRU to forecast short-term electricity demand in Pakistan. The system integrates Explainable AI (XAI) through SHAP and LIME to make model predictions transparent and actionable, and features a smart RAG-based Q&A agent for natural language interaction with forecast data.

C.2.2. Inner Panel Key Features

Accurate Forecasting: Ensemble CNN+LSTM+GRU model achieving $R^2 = 0.934$ on Pakistani household energy data.

Explainable AI: SHAP global feature importance + LIME local instance explanations rendered in real time.

Smart Q&A Agent: Natural language queries powered by Retrieval-Augmented Generation (RAG) and LangChain.

Streamlit Dashboard: Interactive web interface accessible on commodity hardware without specialized infrastructure.

Pakistan-Specific Dataset: Trained on REWDP household data fused with Open-Meteo weather API.

Open Source: All code available on GitHub under an open-source license for academic and research reuse.

C.2.3. Back Panel Team and Contact

Project Team:

M Saqib Masood (Group Leader) Reg. No. Su92-Bscsm-F22-133

M Hamza Reg. No. Su92-Bscsm-F22-130

Laiba Ali Reg. No. Su92-Bscsm-F22-137

Supervisor: Mr. Sohaib Hafeez, Lecturer, Department of Computer Science

Co-Supervisor: Mr. Syed Karar Haider Bukhari, Lecturer, Department of Computer Science

Institution: The Superior University, Lahore, Pakistan

Session: 2022–2026 | FYP ID: FYP-BSCS-F25-06

C.3. Project Abstract Card

The following is the project abstract card distributed at the FYP Exhibition:

Eco Forecast: Smart Energy Consumption Forecasting with Explainable AI Integration

Pakistan's electricity demand is highly volatile due to weather variability, rapid urbanization, and inconsistent household consumption patterns, making traditional forecasting methods unreliable. Eco Forecast addresses this challenge by developing a hybrid deep learning forecasting system that integrates Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU) to produce accurate short-term demand predictions at hourly resolution. The system incorporates Explainable AI (XAI) through SHAP and LIME frameworks, providing both global feature importance and local instance-level explanations to enhance model transparency and stakeholder trust. A Retrieval-Augmented Generation (RAG) powered intelligent Q&A agent enables utility analysts and household users to query forecast insights in natural language. Deployed as a Streamlit web application, the system achieved an ensemble R^2 score of 0.934 and RMSE of 0.328 kWh on the REWDP household energy dataset, representing a 19.1% improvement over the ARIMA baseline. Eco Forecast provides a practical, interpretable, and accessible platform for grid stability planning and energy consumption optimization in Pakistan.

Keywords: Energy Forecasting, Deep Learning, CNN, LSTM, GRU, Explainable AI, SHAP, LIME, RAG, Streamlit, Pakistan

Appendix D: Glossary of Terms

This appendix provides definitions of key technical terms, acronyms, and abbreviations used throughout the Eco Forecast Final Year Project report. It is intended to assist readers who may be unfamiliar with specific terminology in the fields of deep learning, explainable AI, energy systems, and software engineering.

Term / Acronym	Definition
ARIMA	AutoRegressive Integrated Moving Average a classical statistical model for time-series forecasting that combines autoregression, differencing, and moving average components.
CNN	Convolutional Neural Network a deep learning architecture that applies convolutional filters to extract local spatial or temporal patterns from input data.
FAISS	Facebook AI Similarity Search an open-source library for efficient similarity search and clustering of dense vector embeddings, used in the RAG module for document retrieval.
GRU	Gated Recurrent Unit a recurrent neural network architecture that uses update and reset gates to model sequential dependencies in time-series data with fewer parameters than LSTM.
LIME	Local Interpretable Model-Agnostic Explanations an XAI technique that explains individual predictions by approximating the model locally with a simpler interpretable model.
LSTM	Long Short-Term Memory a recurrent neural network architecture with memory cells and input/output/forget gates that capture long-range temporal dependencies in sequential data.
MAE	Mean Absolute Error a metric that measures the average absolute difference between predicted and actual values, expressed in the same units as the target variable (kWh).
NEPRA	National Electric Power Regulatory Authority Pakistan's electricity regulator responsible for licensing, tariff determination, and grid oversight.
R^2	Coefficient of Determination a statistical measure indicating how well the model's predictions explain the variance in the actual data. A value of 1.0 represents perfect prediction.

RAG	Retrieval-Augmented Generation an NLP architecture that combines document retrieval (from a vector store) with a large language model to generate contextually grounded responses to natural language queries.
REWDP	Residential Energy and Weather Data Pakistan the household energy consumption dataset used as the primary training data source in Eco Forecast.
RMSE	Root Mean Square Error the square root of the average squared difference between predicted and actual values. Penalizes large errors more heavily than MAE.
SHAP	SHapley Additive exPlanations an XAI technique rooted in game theory that assigns each feature a contribution value (SHAP value) for a given model prediction, ensuring consistency and local accuracy.
SRS	Software Requirements Specification a formal document describing the functional and non-functional requirements of a software system, following the IEEE 830-1998 standard.
XAI	Explainable Artificial Intelligence a set of methods and techniques that make the outputs of AI systems interpretable and understandable to human users, enhancing trust and transparency.

Bibliography & References

Bibliography & References

A. Books & Foundational Texts

- [1] Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications. (Essential for the CNN/LSTM implementation).
- [2] Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts. (The gold standard for time-series methodology).
- [3] Molnar, C. (2022). Interpretable Machine Learning: A Guide for Making Black Box Models Explainable. (The primary resource for SHAP and LIME logic).

B. Research Papers (Deep Learning & XAI)

- [4] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems (NeurIPS), 30. (The original SHAP paper).
- [5] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. Proceedings of the 22nd ACM SIGKDD International Conference. (The original LIME paper).
- [6] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780. (Foundational paper for the LSTM architecture used in your project).

C. Energy Sector & Local Context (Pakistan)

- [7] National Electric Power Regulatory Authority (NEPRA). (2024). State of Industry Report 2023-24. Government of Pakistan. (Authentic source for energy demand challenges in Pakistan).
- [8] Khan, S., & Ali, A. (2022). Deep Learning Applications in Pakistan's Smart Grid: Challenges and Opportunities. Pakistan Journal of Engineering and Technology. (Contextual research for your local gap analysis).

D. Software, Frameworks & Technical Documentation

- [9] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://www.tensorflow.org/>
- [10] Streamlit Documentation. (2025). Streamlit: The fastest way to build and share data apps. <https://docs.streamlit.io/>
- [11] LangChain. (2024). Retrieval-Augmented Generation (RAG) Framework. <https://python.langchain.com/>